

Programming Every Example

像专家一样规模化提升预训练数据质量

中文全文翻译

Fan Zhou, Zengzhi Wang, Qian Liu, Junlong Li, Pengfei Liu
第 42 届国际机器学习大会 (ICML 2025) , PMLR 267: 79281-79327

Programming Every Example: Lifting Pre-training Data Quality like Experts at Scale

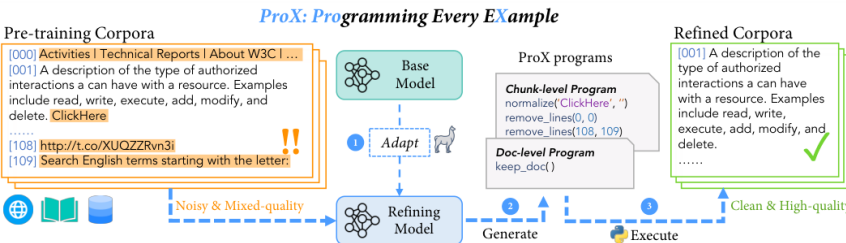


Figure 2: An overview of PROX framework: (1) adapt a base language model for data refinement; (2) generate elaborate programs for each document by PROX refining models, including document-level filtering and more fine-grained chunk-level refining; (3) execute the programs with the docs via a Python executor, producing the refined high-quality corpora.

Table 1: PROX program design of document and chunk level refining stage. doc and chunk will also be sent into the corresponding functions as inputs for execution.

ing specific lines with `remove_lines()` and replacing strings with `normalize()`, providing flexibility in modifying content rather than dropping the whole document.

原论文 Figure 2: ProX 框架概览

非官方中文译本，仅供学习、研究与内部技术交流

译者说明

- 本译本以 ICML 2025 / PMLR 正式版本为底本，按原文结构完整呈现摘要、正文、影响声明、致谢、参考文献和附录。
- 方法名、模型名、数据集名、基准名、数学符号、函数名和代码原则上保留英文；token 统一译为“词元”，corpus 译为“语料库”，refinement 译为“精修”。
- 公式编号、表格数值、训练设置和实验结果按原文保留。主要表格已中文化重排；附录中大规模纯数值表保留原表高分辨率图像，并提供中文表题和术语说明，以避免转录错误。
- 图中英文标签保留原图，同时在图注和正文中给出中文解释。参考文献书目信息保留英文。
- 原文个别拼写或排版问题（如 Programmimg）不在译文中擅自修正实验含义；涉及原文可能存在的技术表述问题，译文忠实保留。

底本信息

Zhou et al. (2025), Programming Every Example: Lifting Pre-training Data Quality Like Experts at Scale, Proceedings of the 42nd International Conference on Machine Learning, PMLR 267, pp. 79281-79327.

题目与作者信息

为每个样本编程：像专家一样规模化提升预训练数据质量

Fan Zhou^{*12}, Zengzhi Wang^{*12}, Qian Liu³, Junlong Li¹, Pengfei Liu ‡¹²⁴

* 共同第一作者。¹ 上海交通大学；² 生成式人工智能研究实验室（GAIR）；³ Sea AI Lab；⁴ 上海人工智能实验室。通讯作者：Pengfei Liu。

摘要

大语言模型预训练长期依赖人类专家编写启发式规则来提高语料库质量，迄今已经形成了大量规则。然而，这些固定规则缺乏灵活性，难以有效处理每个样本的独特特征；而让人类专家为每个样本单独编写规则又不切实际。本文表明，即使只有 0.3B 参数的小型语言模型，也能够表现出可观的数据精修能力。我们提出 Programming Every Example (ProX)，这是一种把数据精修视为编程任务的新框架，使模型能够针对每个样本生成并执行细粒度操作，例如字符串规范化，从而在大规模条件下精修语料库。实验表明，在 10 个基准上，使用 ProX 精修数据训练的模型持续优于其他基线；该方法在不同模型规模（最高 1.7B）和不同预训练语料库（C4、RedPajama-V2、FineWeb、FineWeb-Edu 和 DCLM）上均有效。ProX 在继续预训练中也显示出巨大潜力：在数学领域，仅使用 10B 词元，ProX 就能使 7B 模型的性能最高提升约 20%，而这类结果通常需要更大规模的训练，例如 200B 词元。我们认为，ProX 为构建高质量预训练数据提供了一条路径，并最终有助于提高大语言模型开发效率。

1. 引言

大语言模型（LLM）在能力方面取得了显著进展，在创意写作、复杂推理以及智能体任务规划与执行等任务上表现出色。这些成就背后，大规模、高质量预训练语料库构成了模型的基础，为模型提供完成广泛下游任务所需的知识和推理能力。

互联网提供了海量数据，但其中相当一部分充满噪声、未经精修，在用于预训练之前必须进行大量清洗和质量提升。以往工作主要设计基于启发式规则的流水线，例如文档过滤和基于困惑度的评分方法；这些方案高度依赖人类经验和手工调参。虽然它们得到广泛应用，但劳动密集，受规则覆盖范围限制，也无法处理每一个具体案例。

近期，一些工作开始利用 LLM 获取高质量数据。一方面，语言模型被用于数据过滤或选择，但其作用多局限于识别低质量文档，无法进行字符串级细粒度精修。另一方面，LLM 也被用来直接生成高质量数据，即数据合成。与过滤不同，合成方法会主动创建或改写文档，但需要大量计算资源，限制了可扩展性；同时，它们也会继承 LLM 的幻觉问题，而且难以以可解释方式评估生成数据的正确性与完整性。

在数据处理效率与质量提升的交汇点上，我们提出 ProX，这是一种面向预训练级数据精修的模型化框架。ProX 使用较小模型在规模化条件下精修语料库。实际流程中，ProX 首先用种子数据微调小型基座语言模型（例如小于 1B），使其适应数据精修任务；随后，精修模型通过多样化程序，为预训练语料库中的每篇文档确定合适操作，例如文档过滤、字符串规范化和噪声行删除。生成的程序由预定义执行器执行，得到可直接用于预训练的精修语料库。借助灵活的函数调用，ProX 使语言模型能够自主提升预训练数据质量。

实验表明，ProX 能稳定提高预训练数据质量。在 10 个下游基准上，ProX 相比原始语料库平均提升 2.5 个百分点，并比先进的数据选择方法高出 1.0 个百分点以上。ProX 适用于 0.3B 到 1.7B 的不同模型规模，并在 RedPajama-V2、C4、FineWeb、FineWeb-Edu 和 DCLM 等不同质量的语料库上均带来稳定增益。在领域继续预训练中，经 ProX 精修的 OpenWebMath 在 9 个数学任务上带来显著提升，例如 CodeLlama-7B 提升 20.3 个百分点，并以最高约 20 倍更少的计算达到相近下游结果。定量分析表明，增加用于数据精修的 FLOPs，能够以更低训练成本获得相近性能，为高效 LLM 预训练提供路径。

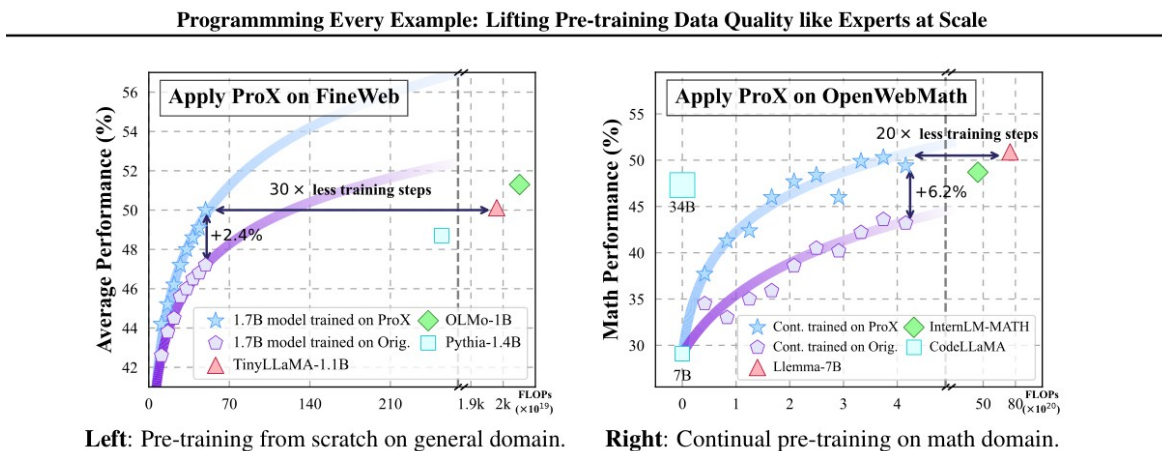


Figure 1: Training FLOPs vs. downstream performance. Although these corpora have been processed through expert-crafted rules, applying PROX still yields significant improvements over these baseline models trained with the original corpora. Moreover, models trained on PROX curated data achieve competitive performance with much fewer training FLOPs.

图 1：训练 FLOPs 与下游性能。左：通用领域从头预训练；右：数学领域继续预训练。即使语料已由专家规则处理，ProX 仍能显著提高性能，并用更少训练 FLOPs 达到有竞争力的结果。

2. 为每个样本编程

2.1 数据精修任务定义

给定语料库 D 中任意文档 d ，例如 HTML 抽取文本或教材，我们把数据精修定义为将 d 转换成质量更高的 $\hat{d}$ 。虽然“更高质量的预训练数据”很难形式化定义，但可以通过定性改善描述，例如删除广告、无意义 URL、随机代码乱码和缺乏教育价值的内容。

具体而言，我们把精修过程表述为：在文档 d 条件下生成一个数据处理程序 Z 。把程序 Z 作用于原始文档 d 后，得到精修文档 $\hat{d}$ 。例如，字符串规范化是一种非常细粒度的过程，可以由执行器 E 和程序 $Z_normalize$ 将噪声字符串转换为干净字符串：

$$E(Z_normalize, d) = (s'_i)_{i=1}^{|d|} \tag{1}$$

$s'_i = \text{normalize}(s_i)$ ，若 s_i 需要规范化；否则 $s'_i = s_i$ 。

其中， $d=(s_1,s_2,...,s_{|d|})$ 表示由字符串序列构成的原始文档， $\text{normalize}()$ 是把特定字符串映射为规范形式的函数。文档过滤是精修转换的特殊情况：执行 Z_filter 会删除整篇文档，即 $E(Z_filter,d)=\emptyset$ 。由此，清洗、规范化等质量提升操作可以统一表示为由 $E(Z,d)$ 执行的标准化函数，其中 Z 编码针对每项具体任务的函数调用或启发式规则。

2.2 ProX 框架

概览。给定任意文档 d ，ProX 使用参数为 θ 的语言模型生成数据精修程序 $Z=f_\theta(d)$ 。程序片段在执行器 E 中运行，得到 $\hat{d}=E(f_\theta(d),d)$ 。ProX 包含两个阶段，从粗到细逐步精修数据：文档级编程和块级编程。每个阶段分别生成 Z_doc 和 Z_chunk ，以不同粒度处理语料库。

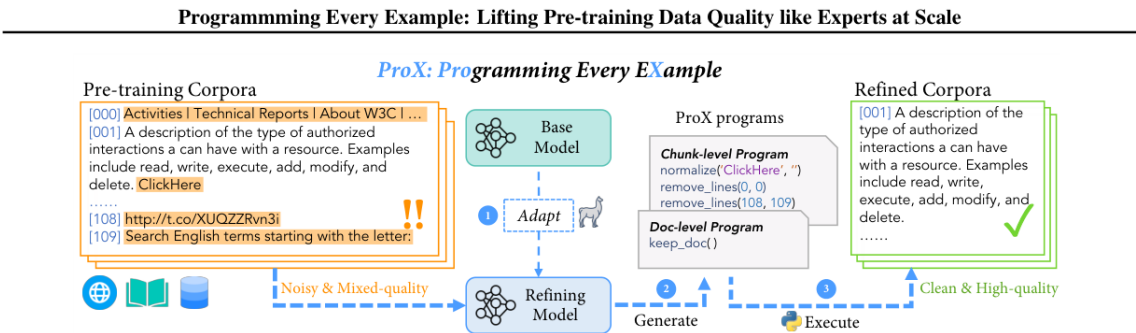


Figure 2: An overview of PROX framework: (1) adapt a base language model for data refinement; (2) generate elaborate programs for each document by PROX refining models, including document-level filtering and more fine-grained chunk-level refining; (3) execute the programs with the docs via a Python executor, producing the refined high-quality corpora.

Table 1: PROX program design of document and chunk level refining stage. doc and chunk will also be sent into the corresponding functions as inputs for execution.

ing specific lines with `remove_lines()` and replacing strings with `normalize()`, providing flexibility in modifying content rather than dropping the whole document.

图 2：ProX 框架总览。（1）把基座语言模型适配到数据精修任务；（2）精修模型为每篇文档生成程序，包括文档级过滤和更细粒度的块级精修；（3）使用 Python 执行器在文档上执行程序，产生高质量精修语料库。

ProX 程序设计。有效的程序空间是最大化语言模型能力的关键。对于大规模预训练语料，作者优先考虑：

(1) 模型足够轻量，能识别特定模式；(2) 虽然比启发式流水线增加计算，但程序设计应保持简单、高效。为了平衡功能与资源，模型只生成函数调用，而不生成函数的详细实现。表 1 中的函数定义构成 ProX 的程序空间。

阶段	函数接口	说明
文档级	<code>drop_doc() → None</code>	删除整篇文档
文档级	<code>keep_doc() → str</code>	返回原始文档
块级	<code>remove_lines(line_start,line_end) → str</code>	删除块中的噪声行；line_start 与 line_end 为起止行索引，包含结束行
块级	<code>normalize(source_str,target_str) → str</code>	把噪声字符串 source_str 替换成 target_str
块级	<code>keep_chunk() → str</code>	返回原始块

文档最基本的操作是删除和替换。文档级阶段使用 `drop_doc()` 删除文档，使用 `keep_doc()` 保留文档。块级阶段把长文档切成较小块，使用 `remove_lines()` 删除指定行，使用 `normalize()` 替换字符串；无需修改的高质量块使用 `keep_chunk()`。单个函数看似简单，但可通过多次调用和不同参数表达复杂的人工规则。作者认为，专家规则能够投影到 ProX 的程序空间中，从而更系统、更可扩展地创建规则。

ProX 执行。在执行阶段，执行器 E 运行生成的程序片段 Z，对文档进行精修。ProX 使用类似 Python 的语法，把操作封装为带参数函数，并用 Python 实现。在图 2 的例子中，文档包含导航栏、无意义 HTML 链接和页面索引等噪声，精修模型生成程序删除相应行和模式。文档级和块级阶段各自使用一个精修模型生成函数调用。顺序处理先解决较大的文档级噪声，再处理更细粒度的局部噪声。

2.3 ProX 模型适配

现成模型通常不能直接生成完美的 ProX 程序。即使当前最强 LLM，生成这种自定义 API 调用也具有挑战，因此需要构建种子数据来适配模型。作者使用先进 LLM，通过 zero-shot 和 few-shot 提示标注操作，再用监督微调 (SFT) 把小模型适配到这些任务。

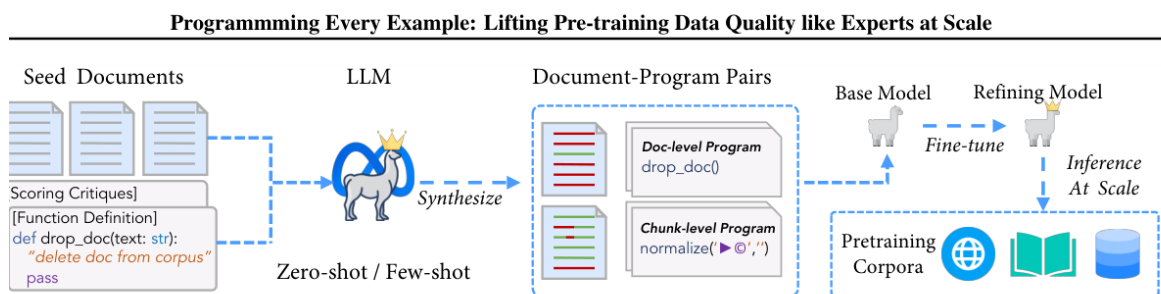


Figure 3: The illustration of the model adaptation in PROX. We employ powerful LLMs (LLAMA-3) to annotate random documents with valid programs and use *doc-program* pairs to fine-tune a small base language model, obtaining the refining model suitable for fine-grained data refining tasks.

图 3：ProX 模型适配。使用强 LLM（Llama-3）为随机种子文档标注合法程序，再用文档-程序对微调小型基座模型，得到可用于细粒度数据精修模型。

作者首先使用加法量表评分提示，把语料划分为保留和丢弃文档，然后用 LLM 为保留文档标注细粒度程序。具体使用 Llama-3-70B 对随机抽取的预训练文档进行标注。标注只进行一次，不同模型都用同一批数据适配。为保证可靠性，作者检查程序语法正确性，并控制删除比例阈值。详细程序合成和后处理见附录 A.1。

为简洁与透明，作者使用在约 26B 原始未精修词元上从头训练的小型基座模型（例如 0.3B），使其训练量超过 Chinchilla 最优点。该模型既作为 ProX 适配的基座，也作为后续实验的比较基线。适配模型在留出的验证集上用 F1 评价；选择最佳 checkpoint 后，为每篇文档或块生成程序 Z，并通过相应函数实现执行，得到最终处理后的语料库。训练细节、F1 计算和大规模推理分别见附录 A.2、A.3 和 A.4。

3. 实验

本节先介绍实验设置（3.1），随后验证 ProX 各精修阶段的有效性并与预训练数据选择方法比较（3.2）；然后在不同模型规模和语料库上验证广泛适用性（3.3）；最后应用于数学领域，验证领域继续预训练中的优势（3.4）。

3.1 实验设置

预训练语料库。通用领域从 RedPajama-V2 开始，它是来自多种互联网来源、约 30T 词元的已预处理数据集。作者还将 ProX 用于约 198B 词元的 C4，以及 FineWeb、FineWeb-Edu 和 DCLM 等近期高质量数据集。领域实验使用约 15B 词元的数学数据集 OpenWebMath。完整语料细节见附录 B.2 表 8。

基座模型。（1）验证 ProX 各阶段时，使用与 Llama-2 架构相同的 0.7B 模型 TLM-S，同时用于从头预训练和精修；与数据选择方法比较时使用 MATES 所采用的 Pythia-410M/1B 架构。（2）评估不同精修模型与基座规模时，把模型扩展到 0.3B 的 TLM-XS 和 1.7B 的 TLM-M。（3）数学领域继续预训练选择 TinyLlama-1.1B、Llama-2-7B、CodeLlama-7B 和 Mistral-7B。模型规格与训练配置见附录 B.3 的表 9、10。

基线。为公平比较，同一实验中的训练超参数基本一致，仅数据精修或选择流水线不同。基线包括：（1）Gopher、C4 和 FineWeb 的启发式规则；（2）在 ProX 种子数据上训练的 FastText 过滤器；（3）DSIR、DsDm、MATES 和 QuRating 等数据选择方法；（4）InstructionLM、Cosmo 等 LLM 数据合成方法和 ShearedLLaMA 模型剪枝。数学领域还与 Llemma、InternLM2-Math 等开源强模型比较。完整基线说明见附录 C。

评测。通用从头预训练使用 LightEval，在 10 项任务上评测；为了公平比较数据选择方法，也使用 LM-Eval-Harness。数学领域整合 9 项任务，报告 few-shot chain-of-thought（CoT）性能。具体基准、样本量和配置见附录 D。

3.2 验证 ProX 的有效性

验证每个 ProX 操作。作者先在 RedPajama-V2 原始数据上训练 TLM-S 约 26B 词元（12.5K 步），作为初始基线。随后依次用该 0.7B 模型微调得到文档级和块级精修模型，进行大规模程序合成与执行，得到 D_doc 和 D_doc+chunk。处理 60B 词元的两阶段合成约需要 192 个 A100 GPU 小时。

方法	ARC-C	ARC-E	CSQA	HellaS	MMLU	OBQA	PIQA	SIQA	WinoG	SciQ	AVG	#Win
Raw	26.1	44.3	29.7	39.1	27.3	29.2	66.9	39.0	52.0	67.4	42.1	0/10
Gopher	25.7	44.0	31.3	40.2	27.3	29.0	66.3	39.0	51.2	68.9	42.3	0/10
C4	25.0	46.0	31.0	40.5	27.1	29.2	68.5	40.5	51.7	66.6	42.6	2/10
FineWeb	25.2	46.8	32.6	39.6	27.2	29.0	66.5	39.4	52.4	69.2	42.8	2/10
规则组合	25.2	43.9	30.0	41.9	27.5	31.0	67.0	39.9	51.9	65.3	42.3	0/10
FastText	26.9	49.9	29.5	39.0	28.5	31.8	64.7	39.6	52.1	70.4	43.3	2/10
ProX-D	26.6	49.7	30.1	40.5	29.4	30.4	66.3	39.0	51.2	71.6	43.5	1/10
ProX-D+C	26.4	51.9	30.9	42.4	29.4	31.6	67.9	40.0	52.2	73.5	44.6	3/10

表 2 及图 4 表明，ProX 在整个训练过程中持续优于基线。相较原始语料，ProX-D+C 的平均分提升 2.5 点，其中 ARC-E 提升 7.6 点，HellaSwag 提升 3.3 点；这些提升也出现在 SIQA、WinoGrande 和 CSQA 等通常波动较大的任务上。相比之下，规则方法的整体增益较小：Gopher 仅提升 0.2，C4 提升 0.5；把三套规则组合起来也没有带来更大平均增益。

INTERNLM2-MATH (Ying et al., 2024). Please refer to for full baseline details.

Evaluation Setup We compare the trained models’ performance over a vast of datasets for comprehensive evaluation: (1) For general pre-training, we evaluate performance across selected tasks using lighteval’s implementation (Fourrier et al., 2023); we have also included LM-eval-harness (Bianan et al., 2024) for fair comparison with data selection methods. (2) For domain-specific continual pre-training evaluation, we integrate nine mathematical related tasks and sort few-shot chain-of-thought (CoT) (Wei et al., 2022) performance. The selected evaluation benchmarks, number of evaluation examples, and full details can be found in §D.

Verifying PROX’s effectiveness

Verifying Effectiveness for Each PROX Operation We conduct a series of experiments to verify the effectiveness of each PROX operation. We begin by training M-S on the RedPajama-V2 raw data for approximately 12.5K tokens (or 12.5K steps) as the initial baseline. Following Wettig et al. (2024) and for convenience, we sequentially apply the document-level and chunk-level refining operations by fine-tuning the 0.7B model itself. We then perform large-scale program synthesis and execution using refining models, resulting in $\mathcal{D}_{\text{doc}}$ and $\mathcal{D}_{\text{doc+chunk}}$. Such large scale synthesis requires approximately 192 A100 GPUs for processing 60B tokens of data. The downstream performance is presented in Table 2, including base models trained on the data produced by PROX refinement methods

图 4：不同训练步数下的下游平均性能。Rule 表示表 2 中表现最好的 FineWeb 规则。

与数据选择方法比较。作者在 C4 与 Pythia 架构下，按 MATES 的设置报告 zero-shot 和 two-shot 性能。即使只使用与数据选择相似的文档级阶段 ProX-D，Pythia-410M 上也比最强选择基线 MATES 高 2.2 和 2.5 点；Pythia-1B 上高 1.0 和 2.0 点。ProX 在 8 个任务中的 7 个上获得最佳结果。完整结果见附录 E.2 表 13。

方法	总 FLOPs (×1e19)	0-shot	2-shot	#Win
Pythia-410M Random	6.4	42.7	43.8	0/8
DSIR	6.4	42.5	43.7	1/8
DsDm	10.7	43.4	44.1	0/8
QuRating	26.4	43.5	44.6	0/8
MATES	8.1	44.0	45.0	0/8

ProX	13.2	46.2	47.5	7/8
Pythia-1B Random	17.7	44.7	45.4	0/8
MATES	20.0	45.8	46.4	1/8
ProX	21.9	46.8	48.4	7/8

3.3 在不同模型规模和语料库上应用 ProX

跨规模。作者在同一 26B 词元上训练 350M、0.7B 和 1.7B 模型，并分别适配到文档级和块级任务。所有规模的文档级 F1 均超过 80%，块级 F1 均超过 75%。用不同规模精修模型生成的数据训练不同规模基座，均比原始数据稳定提升约 2 点。TLM-XS 生成的数据下游表现略好，但保留率较低（23.2%），大模型保留率最高可达 28.8%，说明适度更大的精修模型可能在质量与数量之间取得更有利平衡。

精修模型	文档级 F1	块级 F1	原始词元保留率
XS (0.3B)	82.6	75.2	23.2%
S (0.7B)	81.3	75.6	25.6%
M (1.7B)	83.7	77.3	28.8%

跨语料库。为了验证适用性，作者把同一个 ProX-XS 精修模型直接用于 C4、FineWeb、FineWeb-Edu 和 DCLM，不重新构建种子数据，并从头训练约 50B 词元的 1.7B 模型。10 项平均分分别提高：RedPajama-V2 +2.0、C4 +2.9、FineWeb +2.4、FineWeb-Edu +0.9、DCLM +1.7。

训练效率。作者将 ProX 模型与多种训练路线比较，包括用 3T 词元训练的 TinyLlama-1.1B、由 Llama-2-7B 剪枝并额外训练 50B 词元的 ShearedLLaMA-1.3B，以及使用 LLM 合成数据的 InstructionLM-1.3B 和 Cosmo-1.8B。在 FineWeb 上，ProX 精修语料训练的 TLM-M 与这些依赖模型剪枝或远大训练量的模型表现相当；其精修计算开销明显低于大型 LLM 数据合成。

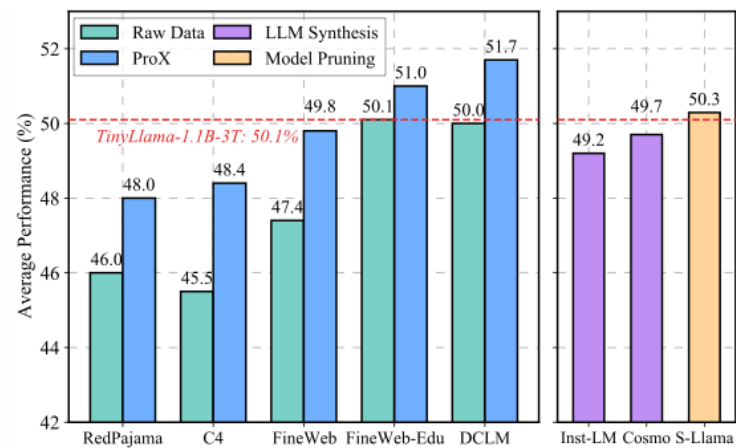


Figure 6: Performance of original data and PROX curated data trained models across different datasets using ≈ 50 B tokens and comparison with existing models trained using different techniques like LLM data synthesis and direct model pruning. Inst-LM: INSTRUCTIONLM-1.3B; Cosmo: COSMO-1.8B; S-Llama: SHEAREDLLAMA-1.3B.

rely heavily on data synthesis with LLMs, underscoring the PROX’s efficiency. Notably, models like INSTRUCT-LM-1.3B, trained on 100 billion tokens from a fine-tuned MISTRAL-7B synthesizer, and COSMO-1.8B, trained on 180B tokens (including 25B tokens synthesized by MISTRAL

图 6：约 50B 训练词元下，不同语料的原始数据与 ProX 数据表现，以及与 LLM 数据合成、模型剪枝路线的比较。

3.4 将 ProX 应用于特定领域

作者在数学领域继续预训练中验证 ProX。对 OpenWebMath 使用与通用领域完全相同的流水线，并用数学种子数据适配 ProX-XS 系列的文档级和块级模型。文档级精修后约保留 5.5B 词元，块级精修后约保留 4.7B。

模型	规模	CPT	方法	独特词元	训练词元	数学平均分
Llemma	7B	是	-	55B	200B	50.9 (+21.8)
InternLM2	7B	否	-	-	-	36.1
InternLM2	7B	是	-	31B	125B	48.7 (+12.6)
TinyLlama	1.1B	否	-	-	-	14.7
TinyLlama	1.1B	是	普通	15B	15B	21.5 (+6.8)
TinyLlama	1.1B	是	规则	6.5B	15B	18.4 (+3.7)
TinyLlama	1.1B	是	ProX	5B	15B	25.7 (+11.0)
Llama-2	7B	否	-	-	-	31.5
Llama-2	7B	是	普通	15B	10B	42.8 (+11.3)
Llama-2	7B	是	ProX	5B	10B	46.1 (+14.6)
CodeLlama	7B	否	-	-	-	29.1
CodeLlama	7B	是	普通	15B	10B	43.2 (+14.1)
CodeLlama	7B	是	ProX	5B	10B	49.4 (+20.3)
Mistral	7B	否	-	-	-	51.6
Mistral	7B	是	普通	15B	10B	54.8 (+3.2)
Mistral	7B	是	ProX	4.7B	10B	59.2 (+7.6)

在不做领域专用程序设计的情况下，ProX 精修 OpenWebMath 使 TinyLlama-1.1B、Llama-2、CodeLlama 和 Mistral 的平均分分别提升 11.0、14.6、20.3 和 7.6 点，超过所有对应基线。规则过滤没有提升 TinyLlama，反而较原始继续预训练下降 3.1 点，说明通用启发式规则并不能适用于所有领域。

与通常需要数百亿至数千亿词元的数学继续预训练模型相比，ProX 显著提高效率。Llemma-7B 基于 CodeLlama-7B，在 200B 词元上训练，平均分为 50.9；ProX 同样从 CodeLlama-7B 出发，仅训练 10B 词元即达到 49.4，训练计算约减少 20 倍。

4. 分析

对原始数据的影响。图 7 比较了原始语料与 ProX 精修语料的文档词元长度分布。在通用领域，原始数据中少于 100 词元的文档占很大比例；应用 ProX 后，多数文档超过 200 词元，平均长度从 1217 增加到 2000 以上。OpenWebMath 中没有出现相同变化，原因可能是其来源主要为 Stack Exchange 等论坛和 arXiv 等学术网站，噪声分布与通用网页不同。进一步分析和案例见附录 F。

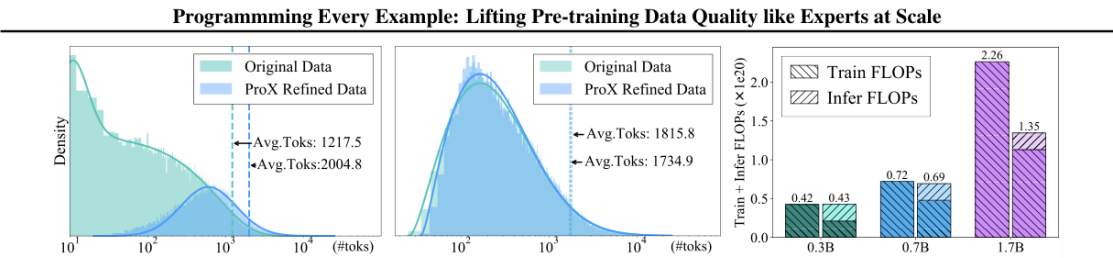


Figure 7: **Left and Middle:** Comparison of doc’s token length distributions between original and PROX-refined data from RedPajama-V2 and OpenWebMath, respectively. **Right:** Total FLOPs comparison for achieving comparable downstream performance with/without PROX refining: 0.3B (Avg. Perf = 40.5), 0.7B (41.6), and 1.7B (42.9).¹

Table 6: Analysis of programs generated by PROX: error program ratio and program complexity (function calls).

Domain	Error Ratio		Avg. Func Calls
	Doc-level	Chunk-level	Chunk-level
General	0.04%	0.36%	3.7
Math	0.06%	0.11%	2.7

PROX Generated Program Analysis PROX maintains extremely low failure rates ($< 0.4\%$) across both refining

(Wenzek et al., 2020; Smith et al., 2022). The remaining documents will then undergo several quality checks with heuristic rules like overall length, symbol-to-word ratio to determine whether they are kept, or aborted (Zhang et al., 2024a; Dou et al., 2024; Qiu et al., 2024). Finally, these documents are deduplicated with algorithms such as Min-Hash (Broder, 1997). In PROX, we use small language models for further data refining, outperforming heuristic rules with acceptable computational overhead.

图 7：左和中：RedPajama-V2 与 OpenWebMath 在 ProX 前后的文档长度分布。右：0.3B、0.7B、1.7B 模型达到相近性能时，有无 ProX 的总 FLOPs。

ProX 生成程序分析。ProX 在两个阶段和两个领域中的失败率都低于 0.4%；通用领域块级程序平均包含 3.7 次函数调用，数学领域为 2.7 次。结果表明，该任务适合小模型，在资源受限条件下也能保持较高可靠性。

领域	文档级失败率	块级失败率	块级平均函数调用
通用	0.04%	0.36%	3.7
数学	0.06%	0.11%	2.7

计算开销分析。大规模 ProX 推理会增加计算预算，但随着待开发模型变大，ProX 推理 FLOPs 在总成本中的相对占比下降。作者比较了有无 ProX 时达到相近下游性能的 checkpoint。对于最大的 1.7B 模型，ProX 方案只使用原始数据基线约 58% 的 FLOPs 即达到相近表现。这说明将额外资源投入预训练数据精修，可能同时提高质量和整体训练效率。

5. 相关工作

预训练数据处理。由于互联网原始数据噪声会损害模型性能，训练前进行预处理已成为常规做法。流水线通常从 URL 过滤、文本抽取等文档准备开始；之后根据整体长度、符号与词语比例等启发式质量规则决定保留或丢弃，最后使用 MinHash 等算法去重。ProX 在这些步骤之后使用小型语言模型进一步精修数据，以可接受的计算开销取得优于启发式规则的效果。

数据选择方法。数据选择通常用于大规模数据预处理后期。在监督微调中，通常是在保持性能的同时选择小得多的样本子集；近期工作把此类策略扩展到预训练。QuRating 训练评分器按四项质量标准打分；MATES 使用语言模型估计数据影响并动态选择。Llama-3 还使用 Llama-2 作为文本质量分类器。ProX 与这些方法不同，在文档内部提供更细粒度的操作。

6. 结论

本文提出 ProX：使用小型语言模型通过程序生成和执行来精修预训练数据。大量实验表明，在 ProX 精修数据上训练能够显著提高多种下游基准性能，并在不同模型规模和数据集上保持有效。数学领域中，ProX 精修数据也以约 20 倍更少的训练词元带来显著提升。进一步分析表明，在大规模 LLM 预训练中，ProX 能以更少计算达到相近结果。ProX 有望在提高数据质量的同时降低 LLM 训练成本；未来进一步扩大用于数据精修的计算，可能加速高效 LLM 的发展。

影响声明

本文工作的目标是推动机器学习领域发展。该工作可能产生多种社会影响，但作者认为没有任何影响需要在此特别强调。

致谢

作者衷心感谢上海人工智能实验室和 Sea AI Lab 提供宝贵计算资源。感谢字节跳动的 Mingxuan Wang 和 Jiaze Chen 提供关键支持；感谢上海交通大学 Ethan Chern、南加州大学 Yuqing Yang 的早期讨论和深刻贡献；感谢加州大学圣迭戈分校 Zhoujun Cheng、香港大学 Yiheng Xu 和 Tianbao Xie、莫纳什大学 Terry Yue Zhuo 提供反馈；感谢 Hugging Face 的 Guilherme Penedo 和 Loubna Ben Allal 指导超参数调优；感谢清华大学 Zhibin Gou 提供继续预训练建议；感谢 Lyumanshan Ye 协助插图和配色设计。特别感谢加州大学圣迭戈分校、TinyLlama 团队的 Peiyuan Zhang 提供优秀的开放预训练框架和一系列加速算子。本项目获得上海交通大学电子信息与电气工程学院-字节跳动大语言模型联合实验室和上海人工智能实验室支持。

参考文献

说明

参考文献的作者、题目、出版信息和 URL 按原文保留英文，不作翻译。以下页面为正式论文参考文献原页。

Programming Every Example: Lifting Pre-training Data Quality like Experts at Scale

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

Acknowledgement

We extend our profound gratitude to Shanghai AI Lab and Sea AI Lab for generously providing valuable computational resources, which were instrumental in the realization of this project. Our sincere thanks also go to Mingxuan Wang and Jiaye Chen from ByteDance for their crucial support. We are deeply thankful to Ethan Chern from Shanghai Jiao Tong University and Yuqing Yang from University of Southern California for their early discussions and insightful contributions, and equally grateful to Zhoujun Cheng from UC San Diego, Yiheng Xu and Tianbao Xie from University of Hong Kong, and Terry Yue Zhuo from Monash University for their valuable feedback, to Guilherme Penedo and Loubna Ben Allal from Hugging Face for their guidance on hyperparameter tuning, to Zhibin Gou from Tsinghua University for providing advice on continual pre-training, to Lyuman-shan Ye for helping with illustrations and color scheme design. Finally, special thanks go to Peiyuan Zhang from UC San Diego, representing the TinyLlama team, for providing a great open pre-training framework and supporting series of acceleration operators. These collective wisdom and unwavering support have been pivotal to our project. This project is supported by SJTU SEIIE - ByteDance Large Language Model Joint Laboratory, Shanghai Artificial Intelligence Laboratory.

References

Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

AI, L. Litgpt. <https://github.com/Lightning-AI/litgpt>, 2023.

Amini, A., Gabriel, S., Lin, S., Koncel-Kedziorski, R., Choi, Y., and Hajishirzi, H. Mathqa: Towards interpretable math word problem solving with operation-based formalisms. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 2357–2367, 2019.

Ankner, Z., Blakeney, C., Sreenivasan, K., Marion, M., Leavitt, M. L., and Paul, M. Perplexed by perplexity: Perplexity-based pruning with small reference models.

In *ICLR 2024 Workshop on Navigating and Addressing Data Problems for Foundation Models*, 2024.

Anthropic, A. The claude 3 model family: Opus, sonnet, haiku. *Claude-3 Model Card*, 2024. URL https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf.

Azerbayev, Z., Schoelkopf, H., Paster, K., Santos, M. D., McAleer, S. M., Jiang, A. Q., Deng, J., Biderman, S., and Welleck, S. Llemma: An open language model for mathematics. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=4WngRR915j>.

Ben Allal, L., Lozhkov, A., Penedo, G., Wolf, T., and von Werra, L. Cosmopedia, February 2024. URL <https://huggingface.co/datasets/HuggingFaceTB/cosmopedia>.

Biderman, S., Schoelkopf, H., Anthony, Q. G., Bradley, H., O'Brien, K., Hallahan, E., Khan, M. A., Purohit, S., Prashanth, U. S., Raff, E., et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pp. 2397–2430. PMLR, 2023.

Biderman, S., Schoelkopf, H., Sutawika, L., Gao, L., Tow, J., Abbasi, B., Aji, A. F., Ammanamanchi, P. S., Black, S., Clive, J., DiPofi, A., Etxaniz, J., Fattori, B., Forde, J. Z., Foster, C., Jaiswal, M., Lee, W. Y., Li, H., Lovering, C., Muennighoff, N., Pavlick, E., Phang, J., Skowron, A., Tan, S., Tang, X., Wang, K. A., Winata, G. I., Yvon, F., and Zou, A. Lessons from the trenches on reproducible evaluation of language models, 2024.

Bisk, Y., Zellers, R., Gao, J., Choi, Y., et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 7432–7439, 2020.

Broder, A. Z. On the resemblance and containment of documents. In *Proceedings. Compression and Complexity of SEQUENCES 1997 (Cat. No. 97TB100171)*, pp. 21–29. IEEE, 1997.

Cheng, D., Gu, Y., Huang, S., Bi, J., Huang, M., and Wei, F. Instruction pre-training: Language models are supervised multitask learners. *arXiv preprint arXiv:2406.14491*, 2024.

Clark, C., Lee, K., Chang, M.-W., Kwiatkowski, T., Collins, M., and Toutanova, K. Boolq: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter*

- of the Association for Computational Linguistics: *Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 2924–2936, 2019.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Dao, T. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.
- Dou, L., Liu, Q., Zeng, G., Guo, J., Zhou, J., Lu, W., and Lin, M. Sailor: Open language models for south-east asia. *CoRR*, abs/2404.03608, 2024. doi: 10.48550/ARXIV.2404.03608. URL <https://doi.org/10.48550/arXiv.2404.03608>.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Engstrom, L., Feldmann, A., and Madry, A. Dsdm: Model-aware dataset selection with datamodels. *arXiv preprint arXiv:2401.12926*, 2024.
- Fan, L., Wang, G., Jiang, Y., Mandekar, A., Yang, Y., Zhu, H., Tang, A., Huang, D.-A., Zhu, Y., and Anandkumar, A. Minedojo: Building open-ended embodied agents with internet-scale knowledge. *Advances in Neural Information Processing Systems*, 35:18343–18362, 2022.
- Fourrier, C., Habib, N., Wolf, T., and Tunstall, L. Lighteval: A lightweight framework for llm evaluation, 2023. URL <https://github.com/huggingface/lighteval>.
- Gunasekar, S., Zhang, Y., Aneja, J., Mendes, C. C. T., Del Giorno, A., Gopi, S., Javaheripi, M., Kauffmann, P., de Rosa, G., Saarikivi, O., et al. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D. d. L., Hendricks, L. A., Welbl, J., Clark, A., et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- Hu, S., Tu, Y., Han, X., He, C., Cui, G., Long, X., Zheng, Z., Fang, Y., Huang, Y., Zhao, W., et al. Minicpm: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., Casas, D. d. l., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35: 22199–22213, 2022.
- Koncel-Kedziorski, R., Roy, S., Amini, A., Kushman, N., and Hajishirzi, H. Mawps: A math word problem repository. In *Proceedings of the 2016 conference of the north american chapter of the association for computational linguistics: human language technologies*, pp. 1152–1157, 2016.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- Li, J., Fang, A., Smyrnis, G., Ivgi, M., Jordan, M., Gadre, S., Bansal, H., Guha, E., Keh, S., Arora, K., et al. Datacomp-llm: In search of the next generation of training sets for language models. *arXiv preprint arXiv:2406.11794*, 2024.
- Li, Y., Bubeck, S., Eldan, R., Del Giorno, A., Gunasekar, S., and Lee, Y. T. Textbooks are all you need ii: phi-1.5 technical report. *arXiv preprint arXiv:2309.05463*, 2023.
- Lin, Z., Gou, Z., Gong, Y., Liu, X., Shen, Y., Xu, R., Lin, C., Yang, Y., Jiao, J., Duan, N., et al. Rho-1: Not all tokens are what you need. *arXiv preprint arXiv:2404.07965*, 2024.
- Liu, J., Cui, L., Liu, H., Huang, D., Wang, Y., and Zhang, Y. Logiqa: A challenge dataset for machine reading comprehension with logical reasoning. *arXiv preprint arXiv:2007.08124*, 2020.
- Liu, R., Wei, J., Liu, F., Si, C., Zhang, Y., Rao, J., Zheng, S., Peng, D., Yang, D., Zhou, D., et al. Best practices and

- lessons learned on synthetic data for language models. *arXiv preprint arXiv:2404.07503*, 2024a.
- Liu, W., Zeng, W., He, K., Jiang, Y., and He, J. What makes good data for alignment? a comprehensive study of automatic data selection in instruction tuning. In *The Twelfth International Conference on Learning Representations*, 2024b. URL <https://openreview.net/forum?id=BTKAeLqLMw>.
- Lu, P., Qiu, L., Chang, K.-W., Wu, Y. N., Zhu, S.-C., Rajpurohit, T., Clark, P., and Kalyan, A. Dynamic prompt learning via policy gradient for semi-structured mathematical reasoning. In *International Conference on Learning Representations (ICLR)*, 2023.
- Luukkonen, R., Komulainen, V., Luoma, J., Eskelinen, A., Kanerva, J., Kupari, H.-M., Ginter, F., Laippala, V., Muennighoff, N., Piktus, A., et al. Fingpt: Large generative models for a small language. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 2710–2726, 2023.
- Maini, P., Seto, S., Bai, H., Grangier, D., Zhang, Y., and Jaitly, N. Rephrasing the web: A recipe for compute and data-efficient language modeling. *arXiv preprint arXiv:2401.16380*, 2024.
- Mehta, S., Sekhavat, M. H., Cao, Q., Horton, M., Jin, Y., Sun, C., Mirzadeh, I., Najibi, M., Belenko, D., Zatloukal, P., et al. Openelm: An efficient language model family with open-source training and inference framework. *arXiv preprint arXiv:2404.14619*, 2024.
- Meta. Introducing meta llama 3: The most capable openly available llm to date, 2024. URL <https://ai.meta.com/blog/meta-llama-3>.
- Miao, S.-Y., Liang, C.-C., and Su, K.-Y. A diverse corpus for evaluating and developing english math word problem solvers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 975–984, 2020.
- Mihaylov, T., Clark, P., Khot, T., and Sabharwal, A. Can a suit of armor conduct electricity? a new dataset for open book question answering. In Riloff, E., Chiang, D., Hockenmaier, J., and Tsujii, J. (eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 2381–2391, Brussels, Belgium, October–November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1260. URL <https://aclanthology.org/D18-1260>.
- Muennighoff, N., Rush, A., Barak, B., Le Scao, T., Tazi, N., Piktus, A., Pyysalo, S., Wolf, T., and Raffel, C. A. Scaling data-constrained language models. *Advances in Neural Information Processing Systems*, 36:50358–50376, 2023.
- Park, J. S., O’Brien, J., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–22, 2023.
- Paster, K., Santos, M. D., Azerbayev, Z., and Ba, J. Openwebmath: An open dataset of high-quality mathematical web text. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=jKHmjlpViu>.
- Patel, A., Bhattamishra, S., and Goyal, N. Are nlp models really able to solve simple math word problems? In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 2080–2094, 2021.
- Penedo, G., Kydliček, H., Lozhkov, A., Mitchell, M., Raffel, C., Von Werra, L., Wolf, T., et al. The fineweb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024a.
- Penedo, G., Kydliček, H., Cappelli, A., Sasko, M., and Wolf, T. Datatrove: large scale data processing, 2024b. URL <https://github.com/huggingface/datatrove>.
- Qiu, J., Lv, H., Jin, Z., Wang, R., Ning, W., Yu, J., Zhang, C., Chu, P., Qu, Y., Peng, R., et al. Wanjian-cc: A safe and high-quality open-sourced english webtext dataset. *arXiv preprint arXiv:2402.19282*, 2024.
- Rae, J. W., Borgeaud, S., Cai, T., Millican, K., Hoffmann, J., Song, F., Aslanides, J., Henderson, S., Ring, R., Young, S., et al. Scaling language models: Methods, analysis & insights from training gopher. *arXiv preprint arXiv:2112.11446*, 2021.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21 (140):1–67, 2020.
- Reid, M., Savinov, N., Teplyashin, D., Lepikhin, D., Lillcrap, T., Alayrac, J.-b., Soricut, R., Lazaridou, A., Firat, O., Schrittwieser, J., et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Canton-Ferrer, C., Grattafiori, A., Xiong, W., Défossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., and Synnaeve, G. Code llama: Open foundation models

- for code. *CoRR*, abs/2308.12950, 2023. doi: 10.48550/ARXIV.2308.12950. URL <https://doi.org/10.48550/arXiv.2308.12950>.
- Sachdeva, N., Coleman, B., Kang, W.-C., Ni, J., Hong, L., Chi, E. H., Caverlee, J., McAuley, J., and Cheng, D. Z. How to train data-efficient llms. *arXiv preprint arXiv:2402.09668*, 2024.
- Sakaguchi, K., Bras, R. L., Bhagavatula, C., and Choi, Y. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- Sap, M., Rashkin, H., Chen, D., LeBras, R., and Choi, Y. Socialiqa: Commonsense reasoning about social interactions. *arXiv preprint arXiv:1904.09728*, 2019.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Zhang, M., Li, Y., Wu, Y., and Guo, D. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Smith, S., Patwary, M., Norick, B., LeGresley, P., Rajbhandari, S., Casper, J., Liu, Z., Prabhunoy, S., Zerveas, G., Korthikanti, V., et al. Using deepspeed and megatron to train megatron-turing nlG 530b, a large-scale generative language model. *arXiv preprint arXiv:2201.11990*, 2022.
- Soldaini, L., Kinney, R., Bhagia, A., Schwenk, D., Atkinson, D., Authur, R., Bogin, B., Chandu, K., Dumas, J., Elazar, Y., Hofmann, V., Jha, A., Kumar, S., Lucy, L., Lyu, X., Lambert, N., Magnusson, I., Morrison, J., Muenighoff, N., Naik, A., Nam, C., Peters, M., Ravichander, A., Richardson, K., Shen, Z., Strubell, E., Subramani, N., Tafjord, O., Walsh, E., Zettlemoyer, L., Smith, N., Hajishirzi, H., Beltagy, I., Groeneveld, D., Dodge, J., and Lo, K. Dolma: an open corpus of three trillion tokens for language model pretraining research. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15725–15788, Bangkok, Thailand, August 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.acl-long.840>.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- Talmor, A., Herzig, J., Lourie, N., and Berant, J. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In Burstein, J., Doran, C., and Solorio, T. (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Volume 1 (Long and Short Papers), pp. 4149–4158, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1421. URL <https://aclanthology.org/N19-1421>.
- Team, I. Internlm: A multilingual language model with progressively enhanced capabilities, 2023.
- Together. Redpajama: an open dataset for training large language models, October 2023. URL <https://github.com/togethercomputer/RedPajama-Data>.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023a.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023b.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Welbl, J., Liu, N. F., and Gardner, M. Crowdsourcing multiple choice science questions. *arXiv preprint arXiv:1707.06209*, 2017.
- Wenzek, G., Lachaux, M., Conneau, A., Chaudhary, V., Guzmán, F., Joulin, A., and Grave, E. Ccnet: Extracting high quality monolingual datasets from web crawl data. In Calzolari, N., Béchet, F., Blache, P., Choukri, K., Cieri, C., Declerck, T., Goggi, S., Isahara, H., Maegaard, B., Mariani, J., Mazo, H., Moreno, A., Odijk, J., and Piperidis, S. (eds.), *Proceedings of The 12th Language Resources and Evaluation Conference, LREC 2020, Marseille, France, May 11-16, 2020*, pp. 4003–4012. European Language Resources Association, 2020. URL <https://aclanthology.org/2020.lrec-1.494/>.
- Wettig, A., Gupta, A., Malik, S., and Chen, D. QuRating: Selecting high-quality data for training language models. In *International Conference on Machine Learning (ICML)*, 2024.
- Xia, M., Gao, T., Zeng, Z., and Chen, D. Sheared llama: Accelerating language model pre-training via structured pruning. In *The Twelfth International Conference on Learning Representations*, 2024.

- Xie, S. M., Santurkar, S., Ma, T., and Liang, P. S. Data selection for language models via importance resampling. *Advances in Neural Information Processing Systems*, 36: 34201–34227, 2023.
- Ying, H., Zhang, S., Li, L., Zhou, Z., Shao, Y., Fei, Z., Ma, Y., Hong, J., Liu, K., Wang, Z., et al. Internlm-math: Open math large language models toward verifiable reasoning. *arXiv preprint arXiv:2402.06332*, 2024.
- Yu, Z., Das, S., and Xiong, C. Mates: Model-aware data selection for efficient pretraining with data influence models. *arXiv preprint arXiv:2406.06046*, 2024.
- Yuan, A., Coenen, A., Reif, E., and Ippolito, D. Wordcraft: story writing with large language models. In *27th International Conference on Intelligent User Interfaces*, pp. 841–852, 2022.
- Yuan, W., Pang, R. Y., Cho, K., Sukhbaatar, S., Xu, J., and Weston, J. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019.
- Zhang, G., Qu, S., Liu, J., Zhang, C., Lin, C., Yu, C. L., Pan, D., Cheng, E., Liu, J., Lin, Q., et al. Map-neo: Highly capable and transparent bilingual large language model series. *arXiv preprint arXiv:2405.19327*, 2024a.
- Zhang, P., Zeng, G., Wang, T., and Lu, W. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024b.
- Zhao, Y., Gu, A., Varma, R., Luo, L., Huang, C.-C., Xu, M., Wright, L., Shojanazeri, H., Ott, M., Shleifer, S., Desmaison, A., Balioglu, C., Damania, P., Nguyen, B., Chauhan, G., Hao, Y., Mathews, A., and Li, S. Pytorch fsdp: Experiences on scaling fully sharded data parallel. *Proc. VLDB Endow.*, 16(12):3848–3860, aug 2023. ISSN 2150-8097. doi: 10.14778/3611540.3611569. URL <https://doi.org/10.14778/3611540.3611569>.
- Zheng, Y., Zhang, R., Zhang, J., Ye, Y., and Luo, Z. Llmfactory: Unified efficient fine-tuning of 100+ language models. *arXiv preprint arXiv:2403.13372*, 2024.
- Zhuo, T. Y., Vu, M. C., Chim, J., Hu, H., Yu, W., Widyasari, R., Yusuf, I. N. B., Zhan, H., He, J., Paul, I., et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.

附录 A：ProX 实现细节

A.1 监督微调数据收集

本节说明用于生成模型适配 SFT 数据的详细提示词。原则上，通用领域语料（C4、RedPajama-V2、FineWeb）和数学语料 OpenWebMath 使用同一类提示词；所有种子数据均从原始语料随机抽样。

文档级编程。在合成“文档-程序”对之前，作者使用两个 zero-shot 评分提示对每个网页进行评价并组合分数。其中一个与 FineWeb-Edu 相同，用于判断教育价值；ProX 新增格式评分，关注文档格式和结构。两种提示都采用加法量表，模型给出简短评价并在 0-5 分之间打分。

FineWeb-Edu 只保留教育分大于 2 的文档，这在希望保留更多词元时过于激进，例如从 FineWeb 原始 15T 词元中只保留 1.3T。ProX 通过加入格式分放宽规则：

条件	决策
教育分 ≥ 3	保留文档
教育分 $=2$ 且格式分 ≥ 4	保留文档
教育分 < 2	丢弃文档

最终使用 Llama-3-70B-Instruct 标注 51K 条数据，其中 5K 用作验证。作者指出，早期实验中几千条（约 5K）数据已足以赋予模型“编程”能力；扩大到 50K 以上主要为了增强跨文档鲁棒性。

块级编程。作者发现三类噪声在所有语料中反复出现：（1）文档顶部的菜单和导航栏；（2）按钮、HTML 元素和链接；（3）页脚。一般情况下，给出 5 个以内示例，LLM 就能工作良好。作者为每种噪声使用 Llama-3-70B-Instruct 进行 few-shot 程序生成，再合并不同程序、检查语法，形成块级训练和验证数据。数据来源与文档过滤阶段相同，总计约 57K，其中 5K 用作验证。

Llama-3.1-405B 发布后，作者尝试用单一 prompt 删除所有噪声，但发现会过度删除原文并降低连贯性。最终只保留 405B 生成程序的头部和尾部，并与先前 Llama-3-70B 生成的程序合并。

教育价值评分提示词（中文翻译）

下面给出一个网页摘录。请使用以下加法式五分评分体系，评价该网页是否具有较高教育价值，是否适合从小学到中学阶段的教学。满足每项标准时累加分数：

- 若摘录包含与教育主题相关的基础信息，即使包含广告、推广等无关或非学术内容，加 1 分。
- 若摘录涉及部分教育相关元素，但与教育标准不够一致；可能混合教育和非教育内容、只对有用主题作表面概述，或组织混乱、写作不连贯，再加 1 分。
- 若摘录适合教育用途，并介绍学校课程相关关键概念；内容连贯，但可能不全面或含少量无关信息；可能类似教材导言或基础教程，但存在明显限制，例如概念对中小學生过于复杂，授予第 3 分。
- 若摘录对不高于中学阶段的教学高度相关且有益，写作清晰一致；类似教材章节或教程，包含大量教育内容、练习与答案，无关信息很少，概念不过度超纲，授予第 4 分。
- 若摘录教育价值出色，非常适合小学或中学教学；推理细致、易于理解，对主题提供深入全面的见解，且没有非教育或过于复杂内容，授予第 5 分。

网页摘录: <EXAMPLE>。检查后: (1) 在 100 词以内简要说明总分理由; (2) 以 “Educational score: <总分>” 格式结尾。

格式质量评分提示词 (中文翻译)

请评价所给网页内容抽取样本。根据满足标准逐项累加:

- 0 分: 从 0 分开始。
- 1 分: 包含一些可读内容, 即使存在大量 HTML 标签、导航元素或其他网页残留; 主要内容可识别, 但与噪声混合。
- 2 分: 出现基础清洗迹象, 大部分明显 HTML 标签已删除, 但可能仍有残留; 文本结构开始显现, 但页脚链接、按钮文字等非正文仍可能存在, 写作可能因页面结构残留而割裂。
- 3 分: HTML 和大多数非正文元素已基本清除, 正文主体完整连贯; 可能仍有孤立 URL、时间戳、图片替代文本等, 但不明显影响可读性, 像原内容的粗略草稿。
- 4 分: 抽取高度精修, 段落结构和格式清晰, 几乎所有 HTML 与非正文元素已清除, 仅有极少噪声; 内容流畅, 格式和风格一致, 接近终稿。
- 5 分: 抽取无瑕疵, 内容完全干净, 并保留原始结构 (段落、标题、列表), 没有 HTML 或网页元素, 也没有任何无关信息; 读起来像专业编辑完成的文档。

网页摘录: <EXAMPLE>。检查后: (1) 在 100 词以内简要说明总分理由; (2) 以 “Extraction Quality Score: <总分>” 格式结尾。

与 FineWeb-Edu 比较。FineWeb-Edu 从 15T 词元中保留约 1.3T (不足 9%), ProX 通常保留 23%-28%, 最多提供约 3 倍独特训练词元。初步 0.7B 实验中, 两种数据的平均性能均为 45.2, 因此作者认为 ProX 更适合大规模预训练。

方法	保留率	ARC-C	ARC-E	CSQA	HellaSwag	MMLU	OBQA	PIQA	SIQA	WinoG	SciQ	AVG	#Win
FineWeb-Edu	8.6%	30.3	58.7	29.0	42.0	30.4	31.8	67.7	38.1	50.4	73.3	45.2	5/10
FineWeb-ProX	28.0%	27.7	55.7	30.4	44.2	29.5	31.0	68.8	39.3	52.2	72.8	45.2	5/10

导航栏删除提示词 (中文翻译)

任务: 生成 Python 程序, 通过删除导航栏来清洗网页文本。文本按从 [000] 开始的行号提供。使用以下预定义函数:

```
def untouch_doc():
    """保持干净文档不变, 用于标记干净且高质量的文档。"""

def remove_lines(start: int, end: int):
    """删除从 start 到 end (包含 end) 的噪声行。"""
```

目标: 识别文本开头的导航栏或菜单项, 并用 remove_lines() 删除。若没有导航栏或菜单, 调用 untouch_doc()。如果一行除导航外还包含其他文本, 也应调用 untouch_doc(), 避免过度删除。示例包括: 删除 “Home | Products | About Us | Contact” 整行; 法律判例标题、新闻标题和分类标签不应误判为导航。

对每段网页文本：（1）简要说明开头约 10 行是否含导航栏；（2）不能把页面标题误认为导航并删除；（3）整行是导航时调用 `remove_lines`，若一行还含其他信息则使用 `normalize` 只删除局部；（4）按 `python [your code]` 输出程序。

URL 删除提示词（中文翻译）

任务：生成 Python 程序，删除网页文本中的 HTTP 链接。除 `untouch_doc` 和 `remove_lines` 外，还提供：

```
def normalize(source_str: str, target_str: str=''):
    """把噪声字符串转换为规范字符串。"""
```

若整行都是 HTTP 链接，使用 `remove_lines`；若链接与正文在同一行，只使用 `normalize` 删除链接而不触碰正文；若没有 HTTP 行，使用 `untouch_doc`。程序只能包含函数调用和注释；行号从 0 开始，必须填写给定行的精确整数，不得加减 1。

```
# 整行是链接
remove_lines(start=13, end=13)
remove_lines(start=15, end=15)

# 链接与正文混合，只删除链接
normalize(source_str='http://groups.google.com/group/toowoombalinuxLast', target_str='')
normalize(source_str='http://www.jitsi.org/.', target_str='')
```

页脚删除提示词（中文翻译）

任务：生成 Python 程序，删除网页文本中的页脚和参考信息。页脚通常位于文本末尾，可能包含版权声明、联系方式或导航链接。若不存在页脚或参考信息，调用 `untouch_doc`。

```
# 从版权声明开始删除页脚
remove_lines(start=14, end=16)

# 从分隔线开始删除订阅页脚
remove_lines(start=22, end=24)
```

对每段文本，简要说明是否有页脚或参考信息，并确保删除不会影响正文。程序只能包含函数调用和注释；行号从 0 开始，必须精确；按 `python [your code]` 输出。

A.2 监督微调细节

作者使用 LLaMA-Factory 作为适配阶段的主要代码库，对基座模型进行全参数监督微调：在全部种子数据上训练 3-5 个 epoch，batch size 64，使用余弦学习率调度，学习率从 1e-5 降到 1e-6。基座模型在这些任务上收敛很快，因此没有进一步搜索超参数，所有适配任务使用同一配置。

A.3 ProX 精修任务的评价指标

文档级精修等价于二分类：文档被分为保留（1）或丢弃（0）。使用 F1 分数：

$F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$	(3)
$\text{Precision} = TP / (TP + FP), \text{Recall} = TP / (TP + FN)$	(4)

F1 范围为 0 到 1，越高表示分类性能越好。

块级任务包含删行和字符串规范化两部分。规范化难以直接评价，因此作者用删行准确性反映精修性能，并提出逐行 F1。先从预测程序和标签程序中提取噪声行索引集合，交集数量为 TP；预测为噪声但标签不是的行为 FP；标签为噪声但预测未删除的行为 FN：

$TP = \text{预测噪声行} \cap \text{实际噪声行} $	(5)
$FP = \text{预测噪声行} \setminus \text{实际噪声行} $	(6)
$FN = \text{实际噪声行} \setminus \text{预测噪声行} $	(7)
$F1 = 2TP / (2TP + FP + FN)$	

A.4 ProX 的规模化推理

借助 Datatrove，作者可以把完整语料高效切分并加载到每个 worker；worker 数通常等于 GPU 数，因为小模型不需要张量并行。大规模推理使用 vLLM。

块级编程会把原始文档切成多个块，使每块词元数小于上下文窗口。实践中，为节省时间，作者通常以词数近似词元数，并把窗口大小设为 1500。算法如下：

```
算法 1：文档块切分
输入：文档 D，上下文窗口 W；输出：块集合 C
1 C ← ∅, c ← ∅
2 逐行遍历 l：
3   若 TokenCount(c+l) ≤ W：把 l 加入当前块 c
4   否则：若 c 非空，保存 c；
5         若 TokenCount(l) ≤ W，以 l 开始新块；
6         否则把该超长行标记为 skipped，并将 c 置空
7 遍历结束后，若 c 非空，加入最终块
8 返回 C
```

附录 B：预训练细节

B.1 训练基础设施

作者使用 LitGPT 和 TinyLlama 灵活训练所有基座模型，并继承 TinyLlama 中来自 FlashAttention 的若干融合算子，包括融合 RoPE、层归一化和交叉熵损失，以节省显存。训练大模型时主要使用 FSDP 进行多节点训练。

B.2 预训练语料库

- 由于计算限制及公平比较需要，作者无法遍历完整语料，因此对部分预训练语料随机采样，形成训练数据池：
- RedPajama-V2：随机下载 70 个文件分片，约 500B 词元，均分成 8 个 dump，每个约 62.5B。3.2 节验证使用 1 个 dump，3.3 节扩展到 50B 训练时使用 2 个 dump。
 - C4：下载完整数据集，约 198B 词元。
 - FineWeb：下载官方 350B sample。
 - OpenWebMath：下载完整数据集。

实验	来源/处理	语料大小 B	训练词元 B	Epoch
3.2 表 2/图 4	RedPajama raw	62.5	26.2	0.42
	规则后	31.5	26.2	0.83
	ProX-D 后	19.0	26.2	1.38
	ProX-D+C 后	16.0	26.2	1.64
3.2 表 3	C4 ProX-D 后	41.5 (GPT-NeoX)	26.2	0.63
3.3 图 5	RPJ ProX-XS	14.5	26.2	1.80
	RPJ ProX-S	16.0	26.2	1.64
	RPJ ProX-M	18.0	26.2	1.46
3.3 图 6	C4 raw / ProX	198 / 44.5	52.4	0.53 / 1.18
	RedPajama raw / ProX	123.5 / 29	52.4	0.42 / 1.81
	FineWeb raw / ProX	79 / 18	52.4	0.66 / 2.91
3.4 1.1B	OpenWebMath raw	15.0	15.7	1.05
	规则后	6.5	15.7	2.40
	ProX-D 后	5.5	15.7	2.85
	ProX-D+C 后	4.7	15.7	3.49
3.4 7B	OpenWebMath raw	15.0	10.5	0.70
	ProX-D 后	5.5	10.5	1.91
	ProX-D+C 后	4.7	10.5	2.23

B.3 模型配置与训练参数

从头训练模型包括 TLM-XS、TLM-S、TLM-M 和 Pythia-410M/1B；继续预训练模型包括 TinyLlama-1.1B、Llama-2-7B、CodeLlama-7B、Mistral-7B。完整架构与参数数量见表 9。

模型	Hidden	Intermediate	Context	Heads	Layers	Vocab	参数量
TLM-XS	1280	2048	2048	16	24	32000	354.3M
TLM-S	1536	4864	2048	24	24	32000	759.0M
TLM-M	2048	8192	2048	32	24	32000	1.742B
Pythia-410M	1024	4096	1024	16	24	50304	405.3M
Pythia-1B	2048	8192	1024	8	16	50304	1.012B
TinyLlama-1.1B	2048	5632	2048	32	22	32000	1.100B
Llama-2-7B	4096	11008	4096	32	32	32000	6.738B

CodeLlama-7B	4096	11008	4096	32	32	32016	6.739B
Mistral-7B	4096	14336	4096	32/8 GQA	32	32000	7.242B

通用从头预训练中，TLM-XS 与 TLM-S 学习率为 5e-4，TLM-M 为 3e-4；最大序列长度统一为 2048，全局 batch 为 2M 词元。与 MATES 比较时完全对齐其超参数，并使用 warmup-stable-decay（WSD）学习率：

$$\begin{aligned}
 \text{lr}(t) = & \begin{cases} (t/W) \cdot \eta, & t < W \\ \eta, & W \leq t < S \\ 0.54^{((t-S)/D)} \cdot \eta, & S \leq t < S+D \end{cases} \\
 & \text{其中 } W=2000, S=50000, D=200.
 \end{aligned}
 \tag{8}$$

继续预训练对不同模型采用不同超参数，并对 7B 模型使用 InternLM2-Math 提到的早停机制。设置主要参考 Rho-1 和 Llemma，不使用 warmup。完整参数见原文表 10。

附录 C：ProX 基线选择

- 为公平比较训练成本，除数据精修/选择流水线外，各基线的训练步数、batch size 等超参数尽量保持一致。
- 3.2 节：与 RedPajama-V2 原始预训练、FineWeb 使用的 C4/Gopher/FineWeb 规则组合、在 ProX 种子上复现的 FastText、DSIR、DsDm、MATES 和 QuRating 比较。
 - 3.3 节：把 TLM-M 在 RedPajama-V2、C4、FineWeb 上扩展到约 50B 词元；效率比较包括 TinyLlama-1.1B（3T 词元）、ShearedLLaMA（剪枝后额外 50B）、InstructionLM（Mistral-7B 合成数据）和 Cosmo（Mixtral-8x7B 合成数据）。
 - 3.4 节：数学继续预训练除 TinyLlama、Llama-2、CodeLlama、Mistral 标准 CPT 外，还与 Rho-1、InternLM2-Math、Llemma 和 DeepSeek-Math 中报告的内部 checkpoint 比较。

附录 D：下游任务评价

D.1 通用预训练评价

LightEval 配置主要沿用 FineWeb 的 9 个“早期信号”任务，并加入 SciQ。默认报告归一化 zero-shot accuracy。任务包括 ARC-Easy、ARC-Challenge、CommonSenseQA、HellaSwag、MMLU、OpenBookQA、PIQA、SocialIQA、WinoGrande 和 SciQ。

FineWeb 配置为每个数据集随机抽取 1000 样本；MMLU 的 57 个子集各抽取 1000。平均分把 ARC-C 与 ARC-E 当成两个任务、MMLU 当成一个任务，以避免 MMLU 因子集多而占主导。zero-shot 评测速度快，适合 20 多次最终训练和数百个中间 checkpoint；作者也在表 11 报告 few-shot 结果，并发现 HellaSwag、MMLU、PIQA 和 WinoGrande 并非都因 few-shot 得到明显提升。

与数据选择方法比较时，使用 LM-Eval-Harness，包含 ARC-E/C、HellaSwag、LogiQA、OpenBookQA、PIQA、WinoGrande 和 SciQ，共 8 项。由于 BoolQ 在早期阶段波动严重且呈下降趋势，作者将其排除。若指标提供 normalized accuracy 则使用该指标，否则使用 accuracy；报告 zero-shot 和 two-shot。

D.2 继续预训练评价

使用 math-eval-harness 中的 MATH、GSM8K、SVAMP、ASDiv、MAWPS、MathQA、TableMWP、SAT Math，并在汇总中加入 MMLU-STEM，共 9 项。评价采用 few-shot CoT，报告每项准确率。

附录 E：完整评价结果

数值表呈现说明

附录 E 由大量模型 checkpoint × 基准的数值表构成。为确保每个数值与加粗/下划线标记不发生转录错误，本译本保留正式论文原表高分辨率图像；表题和说明已译为中文。表内模型名、任务缩写和数值按原文保留。

E.1 给出 3.2 节 10 个基准的 few-shot 结果与 TLM-S 各训练步数完整结果；E.2 给出数据选择方法在 8 个基准上的完整比较；E.3 给出不同精修模型尺寸、不同预训练模型尺寸，以及 RedPajama-V2、C4、FineWeb 等语料扩展实验；E.4 给出数学领域继续预训练的完整消融和各 checkpoint 结果。

Table 11: Few-shot performance on 10 selected tasks. All models use the same TLM-s architecture and are trained on RedPajama-V2. The doc-level (PROX-D) and chunk-level (PROX-C) refining are done by fine-tuning the raw data pre-trained model as a refining model same as Table 2.

Method	ARC-C	ARC-E	CSQA	HellaS	MMLU	OBQA	PIQA	SIQA	WinoG	SciQ	AVG
Raw	25.5	50.3	33.2	39.9	27.8	29.2	67.8	38.7	52.4	71.5	43.6
Rule-based	26.2	50.9	34.1	41.8	27.8	29.2	66.8	40.5	52	72.8	44.2
PROX-D	29.1	55.7	35.6	41.8	29.4	29.2	66.8	38.3	51.3	77	45.4
PROX-D+C	27.2	59.9	38.3	42.8	29.7	31.4	67.1	40.3	50.2	75.8	46.3

表 11：10 项任务的 few-shot 性能。所有模型使用 TLM-S 架构并在 RedPajama-V2 上训练；ProX-D/ProX-C 使用与 zero-shot 实验相同的精修模型。

Table 12: Full evaluation results on TLM-S.

Train Steps	ARC-C	ARC-E	CSQA	HellaSwag	MMLU	OBQA	PiQA	SIQA	WinoG	SciQ	AVG
Raw Data											
2500	22.1	39.0	27.6	31.6	25.9	26.6	61.2	37.3	48.9	59.1	37.9
5000	24.4	41.2	28.8	34.8	26.7	27.0	64.9	39.3	50.4	61.9	39.9
7500	26.5	43.9	29.5	37.2	27.2	29.0	64.8	38.7	50.8	68.2	41.6
10000	25.8	43.5	29.1	38.8	27.4	29.8	66.9	39.0	51.2	66.2	41.8
12500	26.1	44.3	29.7	39.1	27.3	29.2	66.9	39.0	52.0	67.4	42.1
Gopher											
2500	22.3	39.4	26.6	31.3	25.6	27.0	61.1	38.9	51.3	58.6	38.2
5000	25.1	41.4	29.8	34.3	26.4	27.2	64.5	39.6	52.1	62.9	40.3
7500	26.5	43.0	30.5	38.5	27.2	28.8	65.7	38.2	53.7	66.4	41.8
10000	26.2	44.2	31.8	39.2	27.5	29.4	66.6	38.9	51.3	68.2	42.3
12500	25.7	44.0	31.3	40.2	27.3	29.0	66.3	39.0	51.2	68.9	42.3
C4											
2500	22.6	40.6	28.8	31.3	26.2	27.4	61.7	39.3	51.2	57.1	38.6
5000	22.9	41.6	29.3	36.0	26.8	27.6	64.7	40.2	50.9	63.6	40.4
7500	24.2	44.2	29.5	39.2	27.2	28.4	66.2	40.9	51.6	63.8	41.5
10000	24.6	44.8	30.4	39.5	27.0	29.4	68.7	40.9	51.7	63.9	42.1
12500	25.0	46.0	31.0	40.5	27.1	29.2	68.5	40.5	51.7	66.6	42.6
FineWeb											
2500	23.2	39.4	27.2	31.8	25.6	26.2	62.6	39.0	51.4	57.1	38.3
5000	24.2	42.3	29.8	36.2	27.0	28.4	64.3	38.9	51.4	61.4	40.4
7500	24.4	44.1	30.4	37.8	27.2	28.2	66.1	39.5	50.8	66.2	41.5
10000	23.6	46.6	32.0	39.6	27.0	27.8	66.3	39.2	53.1	70.5	42.6
12500	25.2	46.8	32.6	39.6	27.2	29.0	66.5	39.4	52.4	69.2	42.8
Gopher + C4 + FineWeb											
2500	23.6	39.3	27.6	32.1	25.8	26.0	61.7	39.8	50.9	55.4	38.2
5000	23.9	40.9	29.0	36.2	26.9	26.8	65.3	39.3	52.7	62.4	40.3
7500	25.6	42.2	30.7	39.7	27.0	28.4	66.0	40.2	51.8	60.9	41.2
10000	25.8	43.3	30.8	41.4	27.5	29.8	66.9	39.5	51.8	63.1	42.0
12500	25.0	43.9	30.0	41.9	27.5	31.0	67.0	39.9	51.9	65.3	42.3
ProX-D											
2500	25.6	43.2	27.7	32.9	27.2	27.0	61.3	39.4	50.6	63.0	39.8
5000	25.4	46.2	28.4	35.7	28.1	28.8	64.7	39.3	53.3	64.2	41.4
7500	26.9	49.2	29.1	39.2	28.6	30.8	65.4	38.8	51.2	71.7	43.1
10000	26.7	48.2	30.5	39.9	28.6	28.6	66.2	39.7	51.9	71.2	43.2
12500	26.6	49.7	30.1	40.5	29.4	30.4	66.3	39.0	51.2	71.6	43.5
ProX-D+C											
2500	24.9	43.4	27.3	32.1	26.9	28.2	60.9	38.8	51.2	60.8	39.5
5000	24.9	49.6	28.8	36.8	27.9	30.6	64.7	38.8	51.1	66.9	42.0
7500	25.5	51.2	30.8	38.8	28.4	31.2	67.3	40.2	50.3	71.7	43.5
10000	26.2	51.7	30.8	39.9	29.0	32.6	68.6	39.7	51.7	73.7	44.4
12500	26.4	51.9	30.9	42.4	29.4	31.6	67.9	40.0	52.2	73.5	44.6

表 12: TLM-S 在不同训练步数上的 10 项完整评价结果。

E.2. Detailed Performance on 8 Benchmarks used in Data Selection Experiments

The full benchmark performance used in data-selection method comparison experiments is presented in Table 13.

Table 13: Detailed evaluation results for different data selection methods.

Method	ARC-C	ARC-E	HellaSwag	LogiQA	OBQA	PIQA	WinoGrande	SciQ	AVG
PYTHIA-410M 0-shot									
Random	25.6	40.2	39.7	24.7	29.4	67.1	50.6	64.1	42.7
DSIR	23.8	39.9	39.6	27.0	28.4	66.8	51.5	63.1	42.5
DsDm	24.7	41.7	40.3	27.5	29	68.1	50.1	65.4	43.4
QuRating	25.4	42.0	40.7	25.3	30.2	67.5	52.1	64.8	43.5
MATES	25.0	41.8	41.0	25.7	30.8	68.7	52.7	66.0	44.0
PROX	27.2	48.9	43.1	26.9	31.8	68.4	54.1	69.5	46.2
PYTHIA-410M 2-shot									
Random	25.3	42.6	39.9	24.1	28.6	66.9	52.2	70.6	43.8
DSIR	23.6	42.0	39.8	26.1	28.6	66.1	51.6	71.4	43.7
DsDm	23.6	44.2	40.1	23.5	29.2	66.5	51.5	74	44.1
QuRating	23.6	43.9	40.4	26.1	30.2	67.4	51.4	74.1	44.6
MATES	25.3	43.8	40.6	24.9	30.6	67.1	53.4	74.1	45.0
PROX	27.0	52.7	42.6	23.7	32.8	68.2	53.9	78.9	47.5
PYTHIA-1B 0-shot									
Random	25.6	43.7	43.8	27.5	31.8	68.9	50.7	65.8	44.7
MATES	25.9	44.9	45.3	28.7	32.2	69.5	52.4	67.3	45.8
PROX	26.2	49.1	46.6	24.8	32.2	70.3	54.2	70.9	46.8
PYTHIA-1B 2-shot									
Random	25.5	45.1	42.9	24.6	30.0	68.3	52.1	74.6	45.4
MATES	26.8	46.1	44.8	25.2	30.6	68.7	51.6	75.7	46.2
PROX	27.3	54.5	46.2	26.6	32.2	69.0	53.9	77.4	48.4

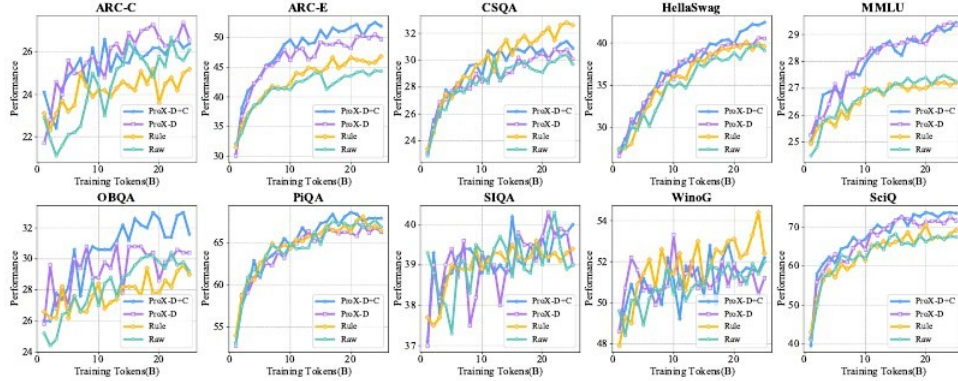


Figure 12: Visualization of dynamic performance on ten benchmarks. Rule: the best performing FineWeb rule in Table 2.

表 13: 不同预训练数据选择方法在 8 项任务上的详细评价结果。

E.3. Detailed Performance in Sec. 3.3

In § 3.3, we test PROX’s effectiveness using different sizes of refining models, and also train a series of models by using these curated data. We report these detailed results in Table 14, Table 15 and Table 16.

Table 14: Full evaluation results of TLM-XS trained on different PROX model curated data.

Train Steps	ARC-C	ARC-E	CSQA	HellaSwag	MMLU	OBQA	PIQA	SIQA	WinoG	SciQ	AVG
TLM-XS trained on Raw data											
2500	22.5	38.5	27.0	29.1	25.8	25.0	60.2	38.8	50.4	58.6	37.6
5000	23.6	39.2	28.7	33.1	26.1	26.6	62.2	39.5	49.9	66.2	39.5
7500	23.8	42.7	28.0	33.4	26.0	26.2	64.0	39.3	51.5	67.0	40.2
10000	23.8	41.2	27.8	35.0	26.6	28.0	65.3	40.9	50.1	65.9	40.5
12500	22.6	41.9	29.7	32.8	26.2	26.4	62.2	39.3	51.3	63.3	39.6
TLM-XS trained on PROX-xs data											
2500	24.8	43.5	26.5	30.3	26.8	26.6	59.3	38.6	50.8	60.7	38.8
5000	23.7	44.3	28.1	33.8	27.3	28.8	61.3	38.9	50.9	70.2	40.7
7500	24.1	46.0	29.2	35.0	27.7	30.6	63.4	38.7	52.0	70.4	41.7
10000	25.3	46.1	28.3	35.7	28.1	29.2	64.4	38.5	51.2	70.6	41.7
12500	25.9	47.5	29.2	36.7	28.1	30.2	64.6	38.0	51.7	71.4	42.3
TLM-XS trained on PROX-s data											
2500	23.5	41.9	24.9	30.4	26.6	27.6	62.0	37.8	49.3	61.4	38.5
5000	24.7	44.5	27.0	33.8	27.5	28.0	62.4	38.0	50.6	67.0	40.3
7500	25.3	45.3	27.3	34.0	27.9	29.2	63.4	37.7	52.9	68.7	41.2
10000	25.6	45.7	27.6	35.6	28.6	30.2	63.6	37.4	52.0	71.1	41.7
12500	26.4	46.7	27.5	37.2	28.1	29.8	62.8	37.8	52.2	70.1	41.9
TLM-XS trained on PROX-m curated data											
2500	22.9	41.3	26.5	31.1	26.9	27.0	62.2	37.6	50.6	62.4	38.9
5000	25.8	44.0	27.3	34.0	27.1	29.6	63.1	38.5	51.8	64.9	40.6
7500	26.0	45.3	28.5	36.6	27.7	29.8	63.6	39.4	51.3	68.5	41.7
10000	26.0	46.6	28.8	37.3	27.6	30.6	63.3	38.7	51.6	70.3	42.1
12500	26.5	46.4	29.1	37.6	28.1	29.4	64.1	38.7	51.5	68.0	41.9

We also further scale PROX to other two pre-training corpora, C4 and FineWeb. We also scale our training to about 50B tokens, and directly compare with existing well-trained models developed by different research groups. We report our detailed results in Table 17, Table 18 and Table 19. We also present other models’ results in Table 20.

表 14：使用不同尺寸 ProX 精修模型所得数据训练 TLM-XS 的完整结果。

Table 15: Full evaluation results of TLM-S trained on different PROX model curated data.

Train Steps	ARC-C	ARC-E	CSQA	HellaSwag	MMLU	OBQA	PiQA	SIQA	WinoG	SciQ	AVG
TLM-S trained on Raw data											
2500	22.1	39.0	27.6	31.6	25.9	26.6	61.2	37.3	48.9	59.1	37.9
5000	24.4	41.2	28.8	34.8	26.7	27.0	64.9	39.3	50.4	61.9	39.9
7500	26.5	43.9	29.5	37.2	27.2	29.0	64.8	38.7	50.8	68.2	41.6
10000	25.8	43.5	29.1	38.8	27.4	29.8	66.9	39.0	51.2	66.2	41.8
12500	26.1	44.3	29.7	39.1	27.3	29.2	66.9	39.0	52.0	67.4	42.1
TLM-S trained on PROX-xs curated data											
2500	23.8	44.1	26.5	33.5	26.9	29.4	60.7	38.9	50.6	62.1	39.6
5000	26.8	48.1	28.4	36.7	28.0	30.6	64.0	38.6	50.3	65.6	41.7
7500	26.9	49.0	30.6	39.5	28.2	29.6	65.3	39.6	52.2	69.6	43.0
10000	26.7	51.3	29.4	40.1	28.3	31.8	64.1	39.3	51.4	69.9	43.2
12500	26.8	52.1	30.2	41.8	28.5	31.6	65.5	39.5	51.9	70.8	43.9
TLM-S trained on PROX-s curated data											
2500	24.9	43.4	27.3	32.1	26.9	28.2	60.9	38.8	51.2	60.8	39.5
5000	24.9	49.6	28.8	36.8	27.9	30.6	64.7	38.8	51.1	66.9	42.0
7500	25.5	51.2	30.8	38.8	28.4	31.2	67.3	40.2	50.3	71.7	43.5
10000	26.2	51.7	30.8	39.9	29.0	32.6	68.6	39.7	51.7	73.7	44.4
12500	26.4	51.9	30.9	42.4	29.4	31.6	67.9	40.0	52.2	73.5	44.6
TLM-S trained on PROX-m curated data											
2500	25.3	45.3	27.5	32.2	26.7	27.0	62.4	38.7	50.6	60.8	39.6
5000	26.1	45.4	28.6	37.2	27.4	27.8	65.7	38.9	50.9	65.6	41.4
7500	27.1	47.5	30.6	41.0	28.6	29.2	66.8	39.3	51.1	69.9	43.1
10000	26.7	50.5	30.7	41.5	28.4	30.2	67.0	40.1	49.9	70.9	43.6
12500	27.4	50.7	30.6	42.0	28.8	30.2	67.4	39.4	48.8	70.1	43.5

表 15：使用不同尺寸 ProX 精修模型所得数据训练 TLM-S 的完整结果。

Table 16: Full evaluation results of TLM-M trained on different ProX model curated data.

Train Steps	ARC-C	ARC-E	CSQA	HellaSwag	MMLU	OBQA	PiQA	SIQA	WinoG	SciQ	AVG
TLM-s trained on Raw data											
2500	23.5	41.5	27.5	32.9	26.4	25.2	62.1	39.4	51.5	65.1	39.5
5000	24.0	42.1	29.6	37.6	27.6	27.2	65.0	39.7	53.2	68.5	41.4
7500	24.3	44.9	28.9	39.3	27.8	27.6	66.4	40.4	51.3	69.2	42.0
10000	24.8	46.1	29.6	41.4	27.9	28.4	67.5	39.8	51.9	70.9	42.8
12500	26.3	46.8	29.0	43.2	28.3	27.8	68.2	40.5	50.7	72.5	43.3
TLM-M trained on ProX-xs curated data											
2500	24.9	49.6	26.5	34.0	27.3	30.4	61.8	37.9	51.3	65.1	40.9
5000	26.7	47.6	28.6	39.7	28.5	31.8	65.4	39.5	50.2	70.7	42.9
7500	27.5	52.1	30.4	41.8	29.6	31.8	67.6	39.6	51.7	75.2	44.7
10000	28.4	54.7	29.8	45.2	30.8	31.8	67.9	39.7	52.0	77.7	45.8
12500	28.8	54.2	29.7	46.5	30.9	31.8	68.2	39.9	51.3	78.3	46.0
TLM-M trained on ProX-s curated data											
2500	25.3	45.7	27.8	34.2	27.8	29.0	64.4	37.5	49.3	66.3	40.7
5000	26.1	49.0	28.8	40.2	29.2	30.8	65.6	39.0	50.5	71.2	43.0
7500	27.7	53.6	31.1	44.1	29.6	34.8	67.6	39.4	52.5	72.2	45.3
10000	27.2	54.0	31.5	45.1	30.3	33.8	67.7	39.7	52.9	74.2	45.6
12500	28.6	56.1	31.8	45.5	30.5	34.4	68.5	39.4	51.3	76.1	46.2
TLM-M trained on ProX-m curated data											
2500	24.7	44.1	25.9	34.8	27.4	27.8	62.9	38.9	49.2	67.0	40.3
5000	27.7	48.0	26.8	40.5	28.5	30.6	67.4	39.4	50.3	69.1	42.8
7500	26.7	51.9	26.7	42.9	29.3	31.4	69.1	40.3	50.4	73.3	44.2
10000	28.4	52.4	27.9	45.0	29.7	32.0	70.2	40.0	51.9	75.4	45.3
12500	28.3	53.7	28.4	45.9	30.1	33.8	70.6	41.1	52.3	72.5	45.7

表 16: 使用不同尺寸 ProX 精修模型所得数据训练 TLM-M 的完整结果。

Table 17: Full evaluation results on scaling pre-training to about 50B tokens on RedPajama-V2.

Train Steps	ARC-C	ARC-E	CSQA	HellaSwag	MMLU	OBQA	PiQA	SIQA	WinoG	SciQ	AVG
TLM-M trained on RedPajama-V2 raw data.											
2500	24.0	42.9	26.6	33.7	25.9	26.0	62.4	39.4	52.3	64.0	39.7
5000	24.3	45.9	26.4	37.4	27.0	27.6	64.1	39.7	49.5	66.2	40.8
7500	25.1	45.3	28.8	40.3	27.1	29.2	66.3	39.1	51.7	66.9	42.0
10000	25.8	49.3	31.5	42.5	28.0	28.8	66.7	39.6	51.5	74.0	43.8
12500	25.3	50.1	30.2	43.0	28.2	30.0	66.6	39.2	51.1	74.2	43.8
15000	26.2	50.3	31.2	44.3	28.8	28.4	68.2	39.8	51.7	76.2	44.5
17500	25.8	51.1	30.8	44.7	29.0	29.6	67.7	39.2	52.6	75.2	44.6
20000	26.7	52.5	31.7	47.2	28.6	30.4	69.0	39.6	53.0	78.2	45.7
22500	27.4	51.7	32.1	47.2	29.3	30.4	69.5	39.5	51.9	78.5	45.7
25000	26.9	51.4	32.4	47.3	29.3	32.2	69.7	39.6	52.1	79.1	46.0
TLM-M trained on PROX refined RedPajama-V2 data.											
2500	24.8	46.8	27.2	33.8	27.3	28.2	61.3	38.6	50.3	65.1	40.3
5000	26.9	49.3	28.5	40.1	28.0	30.6	66.2	39.7	50.2	70.1	43.0
7500	28.5	53.1	29.2	41.7	29.4	33.2	66.9	39.3	53.0	73.0	44.7
10000	28.2	53.5	30.1	43.6	29.8	31.6	68.4	39.6	52.0	75.3	45.2
12500	29.5	55.3	30.2	46.4	30.5	32.2	68.6	40.2	52.6	76.9	46.2
15000	30.0	57.1	30.2	47.6	30.9	33.0	69.5	39.8	52.2	77.8	46.8
17500	31.5	59.6	29.4	49.5	31.6	33.6	69.4	39.8	53.0	78.9	47.6
20000	31.2	61.2	29.4	50.4	31.4	35.2	70.6	40.1	53.7	79.6	48.3
22500	32.0	61.7	30.2	51.4	31.4	34.0	70.0	39.9	53.2	79.5	48.3
25000	31.1	60.7	29.8	51.0	31.7	33.2	70.9	39.2	53.3	79.1	48.0

Table 18: Full evaluation results on scaling pre-training to about 50B tokens on C4.

Train Steps	ARC-C	ARC-E	CSQA	HellaSwag	MMLU	OBQA	PiQA	SIQA	WinoG	SciQ	AVG
TLM-M trained on C4 raw data.											
2500	22.4	39.7	26.8	36.5	26.5	27.6	64.8	40.2	50.1	60.0	39.5
5000	23.9	42.9	27.5	42.3	27.1	29.6	68.2	39.6	50.3	66.6	41.8
7500	25.1	44.8	28.2	45.4	27.1	29.2	70.7	40.7	51.6	66.3	42.9
10000	25.5	46.0	32.3	48.2	27.9	31.6	71.1	39.7	52.3	67.6	44.2
12500	25.8	48.8	30.3	49.7	27.9	31.6	71.2	40.9	52.0	69.4	44.8
15000	26.9	48.0	28.2	50.5	28.5	31.4	71.9	41.1	51.4	69.7	44.8
17500	26.6	48.8	30.3	52.1	28.6	31.2	73.2	41.6	52.0	70.0	45.4
20000	26.3	50.1	29.7	52.5	28.5	32.6	72.3	41.7	52.3	71.0	45.7
22500	25.8	50.7	31.0	52.9	28.8	33.8	73.0	41.6	53.0	71.5	46.2
25000	25.3	48.8	30.1	52.4	28.8	32.2	72.0	40.6	53.6	71.7	45.5
TLM-M trained on PROX refined C4 data.											
2500	24.1	45.9	26.0	37.3	27.2	29.0	66.3	39.8	50.8	65.9	41.2
5000	27.3	50.0	26.6	42.4	28.6	33.8	68.1	40.5	53.0	71.9	44.2
7500	28.3	53.7	27.7	47.7	29.3	35.4	71.1	39.3	54.0	73.1	46.0
10000	30.0	54.3	28.1	50.9	30.0	33.6	71.2	40.6	52.0	74.2	46.5
12500	29.3	56.7	27.5	52.3	30.9	33.8	72.8	39.9	52.5	77.5	47.3
15000	29.6	55.9	28.3	53.9	30.6	35.0	72.9	41.0	53.8	75.8	47.7
17500	30.6	55.5	28.7	53.3	31.2	34.2	73.6	40.4	53.4	76.7	47.8
20000	30.0	57.6	28.3	54.9	31.1	37.2	74.6	40.7	53.6	79.4	48.7
22500	30.1	56.7	28.6	55.2	31.4	37.2	73.8	41.6	53.3	77.7	48.6
25000	31.1	56.0	28.4	55.2	31.1	36.2	74.0	41.0	54.1	76.8	48.4

表 17: RedPajama-V2 上扩展到约 50B 词元的完整结果; 表 18: C4 上约 50B 词元的完整结果。

Table 19: Full evaluation results on scaling pre-training to about 50B tokens on FineWeb.

Train Steps	ARC-C	ARC-E	CSQA	HellaSwag	MMLU	OBQA	PiQA	SIQA	WinoG	SciQ	AVG
TLM-M trained on FineWeb raw data.											
2500	22.9	41.2	28.9	34.3	26.1	27.6	64.8	39.3	52.1	62.8	40.0
5000	25.5	44.5	30.4	39.8	26.9	32.0	68.4	39.2	52.1	67.2	42.6
7500	26.8	45.6	31.4	44.1	27.6	30.2	70.9	38.8	52.2	70.3	43.8
10000	27.2	46.2	31.3	47.2	28.3	31.6	72.1	38.8	53.4	69.0	44.5
12500	26.4	49.2	32.1	48.7	28.7	31.6	71.5	40.1	52.6	74.7	45.6
15000	27.1	49.6	32.8	49.5	28.9	31.0	72.7	39.0	52.3	77.1	46.0
17500	26.4	50.9	33.8	51.3	29.3	31.0	71.9	39.3	53.0	78.0	46.5
20000	27.1	53.1	33.2	51.2	29.6	32.2	73.4	39.7	52.3	76.3	46.8
22500	27.1	51.2	34.9	51.7	29.5	33.4	73.7	40.1	52.4	78.0	47.2
25000	28.5	52.6	33.9	53.2	29.8	32.6	72.9	40.2	53.0	77.1	47.4
TLM-M trained on PROX refined FineWeb data.											
2500	25.8	46.8	27.4	36.1	27.7	28.8	63.9	39.3	51.9	69.1	41.7
5000	28.5	52.1	28.8	43.5	29.3	32.6	66.4	38.7	51.2	71.3	44.2
7500	28.2	52.0	30.6	45.9	29.9	33.0	69.3	39.5	51.7	71.8	45.2
10000	29.3	54.3	30.6	48.5	30.8	33.2	69.7	40.7	50.6	74.4	46.2
12500	28.7	57.8	30.7	48.1	31.1	32.6	72.0	40.4	52.7	77.4	47.2
15000	31.1	59.6	31.9	50.4	31.8	34.4	71.9	40.5	50.8	78.0	48.0
17500	32.6	60.9	31.9	51.5	32.2	33.8	72.3	39.7	52.5	78.9	48.6
20000	33.2	62.5	32.5	51.6	32.4	34.6	72.4	39.7	51.7	80.7	49.1
22500	34.7	63.6	32.9	53.3	32.9	34.8	73.1	40.3	54.2	80.5	50.0
25000	34.4	63.9	32.6	53.0	33.1	34.4	73.1	39.3	52.7	81.5	49.8

Table 20: Detailed evaluation results of existing base models trained on different corpora and trained using different techniques.

ARC-C	ARC-E	CSQA	HellaSwag	MMLU	OBQA	PiQA	SIQA	WinoG	SciQ	AVG
TINYLLAMA-1.1B (trained on 3T tokens)										
31.5	59.0	35.5	57.8	32.8	33.4	72.8	40.0	56.0	82.4	50.1
OLMO-1B (trained on 2T tokens)										
31.4	59.7	38.9	61.9	32.2	38.4	76.1	41.5	53.9	78.8	51.3
PYTHIA-1.4B										
28.7	56.9	34.7	51.7	31.5	36.0	71.8	40.8	55.1	79.3	48.7
PYTHIA-2.8B										
32.9	61.0	36.5	60.4	33.3	35.0	73.5	41.1	57.0	83.1	51.4
SHEAREDLLAMA-1.3B (pruned from LLAMA-2-7B)										
22.4	39.7	29.3	36.0	26.4	28.4	62.6	39.9	52.0	71.4	40.8
SHEAREDLLAMA-1.3B (pruned from LLAMA-2-7B, and further trained on 50B tokens)										
29.0	58.3	34.8	59.6	32.0	35.0	74.6	41.0	56.3	82.3	50.3
INSTRUCTLM-1.3B (LLM data synthesis)										
28.1	57.9	32.5	52.3	30.0	34.0	74.5	39.9	56.1	86.9	49.2
COSMO-1.8B (LLM data synthesis)										
33.4	57.0	31.2	55.1	32.4	35.2	71.4	42.0	54.7	84.4	49.7

表 19: FineWeb 上约 50B 词元的完整结果；表 20: 使用不同语料与训练技术的现有基座模型详细比较。

E.4. Evaluation Results of Continual Pre-training in Sec. 3.4

We provide full ablation results for each base model, as shown in Table 21. We can observe that PROX-D+C consistently improves average performance over PROX-D across various base models. Although the performance gain from PROX-D+C compared to PROX-D is less pronounced than the improvement of PROX-D over continual pre-training on raw OpenWebMath, this is both understandable and expected. PROX-D+C does not significantly reduce the token count beyond the reductions achieved by PROX-D alone. Given the scale of the OpenWebMath corpus, a more aggressive token removal strategy could potentially diminish the diversity of unique tokens below the threshold necessary for robust pre-training. This observation underscores the delicate balance between data refinement and maintaining sufficient linguistic variety for effective language model training, particularly when working with limited-scale corpora.

Table 21: Full ablation results on OpenWebMath Continual Pre-training (CPT). All models are tested using few-shot CoT prompts. LLEMMA and INTERNLM2-MATH are continual pre-trained models from CODELLAMA (Rozière et al., 2023) and INTERNLM2 (Team, 2023) with public available data, respectively. DEEPSEEK-LLM denotes an internal DeepSeek model, and the model trained on OpenWebMath introduced by Shao et al. (2024). Note that the unique tokens and training tokens in the column refer exclusively to the token numbers from math-specific corpora (calculated by corresponding tokenizers). [†]: MQA evaluation of INTERNLM2-BASE is based on an alternative prompt due to non-prediction issues with the original prompt. The **bolded** entries represent the best results within the same base model and CPT experiments.

Model	Size	Method	Uniq Toks	Train Toks	GSM8K	MATH	SVAMP	ASDiv	MAWPS	TAB	MQA	MMLU STEM	SAT MATH	AVG
Existing Continual Pre-training for Reference														
DEEPSEEK-LLM	1.3B	-	-	-	2.9	3.0	-	-	-	-	-	19.5	15.6	-
	1.3B	-	14B	150B	11.5	8.9	-	-	-	-	-	29.6	31.3	-
CODELLAMA (Base)	7B	-	-	-	11.8	5.0	44.2	50.7	62.6	30.6	14.3	20.4	21.9	29.1
	34B	-	-	-	31.8	10.8	61.9	66.0	83.4	51.6	23.7	43.0	53.1	47.3
LLEMMA	7B	-	55B	200B	38.8	17.2	56.1	69.1	82.4	48.7	41.0	45.4	59.4	50.9 (+21.8)
	34B	-	55B	50B	54.2	23.0	67.9	75.7	90.1	57.9	49.8	54.7	68.8	60.1 (+12.8)
INTERNLM2-BASE	7B	-	-	-	27.0	6.6	49.0	59.3	74.8	40.1	20.9 [†]	19.0	28.1	36.1
	20B	-	-	-	50.6	18.8	72.5	75.9	93.9	45.4	33.1	53.7	59.4	55.9
INTERNLM2-MATH	7B	-	31B	125B	41.8	14.4	61.6	66.8	83.7	50.0	57.3	24.8	37.5	48.7 (+12.6)
	20B	-	120B	500B	65.4	30.0	75.7	79.3	94.0	50.9	38.5	53.1	71.9	62.1 (+6.2)
Applying Data Refinement Approaches														
TINYLLAMA (Base)	1.1B	-	-	-	2.8	3.2	10.9	18.0	20.2	12.5	14.6	16.4	21.9	14.7
TINYLLAMA (CPT)	1.1B	-	15B	15B	6.2	4.8	22.3	36.2	47.6	19.3	11.6	20.7	25.0	21.5 (+6.8)
	1.1B	RHO	15B	9B ⁺⁵	7.1	5.0	23.5	41.2	53.8	-	18.0	-	-	-
	1.1B	Rule	6.5B	15B	4.5	2.8	17.5	29.4	39.3	15.1	12.4	19.4	25.0	18.4 (+3.7)
	1.1B	PROX-D	5.4B	15B	9.3	7.4	23.4	41.9	55.6	22.1	14.6	24.1	25.0	24.8 (+10.1)
	1.1B	PROX-D+C	5B	15B	9.0	5.6	23.8	41.9	56.9	22.2	15.6	26.8	31.2	25.7 (+11.0)
LLAMA-2 (Base)	7B	-	-	-	14.1	3.8	39.5	51.6	63.6	30.9	12.5	32.9	34.4	31.5
LLAMA-2 (CPT)	7B	-	15B	10B	29.6	13.6	49.2	61.9	78.4	36.3	31.9	40.5	43.8	42.8 (+11.3)
	7B	PROX-D	5.4B	10B	30.3	16.0	54.2	63.8	79.5	37.3	37.2	44.2	46.9	45.5 (+14.0)
	7B	PROX-D+C	5B	10B	30.6	16.8	50.2	63.7	79.3	37.3	40.1	43.8	53.1	46.1 (+14.6)
CODELLAMA (Base)	7B	-	-	-	11.8	5.0	44.2	50.7	62.6	30.6	14.3	20.4	21.9	29.1
CODELLAMA (CPT)	7B	-	15B	10B	31.1	14.8	51.4	62.1	81.2	33.6	30.4	40.5	43.8	43.2 (+14.1)
	7B	PROX-D	5.4B	10B	38.1	17.0	54.2	67.0	83.1	40.9	39.8	43.7	50.0	48.2 (+19.1)
	7B	PROX-D+C	5B	10B	35.6	17.6	55.8	67.9	82.7	41.3	38.9	42.6	62.5	49.4 (+20.3)
MISTRAL (Base)	7B	-	-	-	40.6	11.4	65.4	68.5	87.0	52.9	32.3	50.0	56.2	51.6
MISTRAL (CPT)	7B	-	15B	10B	44.4	19.2	65.2	69.6	88.4	46.6	43.1	50.8	65.6	54.8 (+3.2)
	7B	PROX-D	5.5B	10B	47.8	24.8	63.5	72.4	88.9	48.3	48.2	54.1	62.5	56.4 (+4.8)
	7B	PROX-D+C	4.7B	10B	51.0	22.4	64.9	72.9	89.2	49.8	53.0	54.2	75.0	59.2 (+7.6)

Besides, we report the detailed dynamic evaluation results of our continual pre-training experiments on OpenWebMath:

⁵RHO-1 only counts the selected tokens that are used for training (loss calculation).

表 21: OpenWebMath 继续预训练的完整消融；所有模型用 few-shot CoT 在 9 个数学任务上评价。

- Tables 22, 23, 24, and 25 present the evaluation results for TINYLLAMA-1.1B.
- Tables 26, 27, and 28 present the evaluation results for LLAMA-2.
- Tables 29, 30, 31 present the evaluation results for CODELLAMA.
- Tables 32, 33, and 34 show the evaluation results for MISTRAL-7B.

Table 22: Full evaluation results of TINYLLAMA-1.1B continual pre-training on OpenWebMath with raw data. Note that about 1B tokens are trained per 500 steps.

Train Steps	GSM8K	MATH	SVAMP	ASDiv	MAWPS	TAB	MQA	MMLU STEM	SAT MATH	AVG
0	2.8	3.2	10.9	18	20.2	12.5	14.6	16.4	21.9	14.7
500	1.9	3.4	16.3	23.9	30.3	13.9	10.3	14.8	18.8	14.8
1000	3.1	2.2	16.6	25.6	32.4	12.5	12.0	16.6	25.0	16.2
1500	2.7	3.0	17.6	28.5	34.5	13.9	8.7	14.1	15.6	15.4
2000	4.5	3.2	16.4	28.5	39.0	15.1	10.2	16.6	34.4	18.7
2500	4.9	3.4	19.3	31.0	39.2	16.0	12.1	18.6	9.4	17.1
3000	4.1	5.2	19.1	32.0	43.0	15.3	9.6	16.1	18.8	18.1
3500	4.9	3.6	19.7	31.4	40.4	18.1	11.3	19.6	15.6	18.3
4000	4.8	4.8	19.5	33.8	44.5	16.4	10.7	19.9	12.5	18.5
4500	5.4	4.8	20.2	35.0	45.2	17.9	12.7	21.0	18.8	20.1
5000	5.5	4.6	22.3	34.6	42.9	16.0	10.6	21.7	28.1	20.7
5500	4.9	5.8	23.6	35.2	44.0	20.4	11.0	21.1	21.9	20.9
6000	6.1	4.4	22.8	36.2	45.4	17.8	12.7	21.4	15.6	20.3
6500	6.3	3.6	23.2	37.3	48.0	19.7	10.3	21.0	18.8	20.9
7000	6.1	4.6	22.2	36.6	46.9	19.4	12.0	21.5	21.9	21.2
7500	6.2	4.8	22.3	36.2	47.6	19.3	11.6	20.7	25.0	21.5

表 22: TinyLlama-1.1B 在 OpenWebMath 原始数据上继续预训练的完整结果；每 500 步约 1B 词元。

Table 23: Full evaluation results of TINYLLAMA-1.1B continual pre-training on OpenWebMath with data after rule-based filtering. Note that about 1B tokens are trained per 500 steps.

Train Steps	GSM8K	MATH	SVAMP	ASDiv	MAWPS	TAB	MQA	MMLU STEM	SAT MATH	AVG
0	2.8	3.2	10.9	18	20.2	12.5	14.6	16.4	21.9	14.7
500	3.4	3.6	13.6	22.5	25.9	13.1	14.2	13.5	28.1	15.3
1000	3.0	2.8	14.1	22.5	27.8	11.4	11.0	16.4	12.5	13.5
1500	3.6	3.2	13.6	24.0	31.2	13.9	9.2	18.0	18.8	15.1
2000	3.5	2.4	15.0	25.1	33.0	12.5	10.6	13.9	15.6	14.6
2500	3.3	1.6	15.0	25.3	33.5	13.7	11.1	18.1	25.0	16.3
3000	3.5	3.0	16.4	25.5	33.4	14.1	10.2	18.4	18.8	15.9
3500	3.2	3.4	17.2	27.0	37.7	14.6	11.2	13.3	25.0	17.0
4000	3.5	3.6	15.6	26.2	36.5	13.4	12.1	15.9	18.8	16.2
4500	4.1	3.8	15.6	27.9	38.2	14.9	11.6	17.1	18.8	16.9
5000	4.2	3.6	18.6	28.7	37.7	14.3	12.7	17.5	21.9	17.7
5500	4.1	3.8	16.3	29.3	38.4	14.7	10.8	17.5	18.8	17.1
6000	4.3	3.6	16.0	28.7	39.1	13.5	12.8	19.5	21.9	17.7
6500	4.2	3.2	16.4	29.5	39.0	15.1	11.7	17.9	21.9	17.7
7000	4.0	4.0	16.2	29.6	37.9	16.0	13.8	17.8	21.9	17.9
7500	4.5	2.8	17.5	29.4	39.3	15.1	12.4	19.4	25.0	18.4

Table 24: Full evaluation results of TINYLLAMA-1.1B continual pre-training on OpenWebMath with data after PROX-D. Note that about 1B tokens are trained per 500 steps.

Train Steps	GSM8K	MATH	SVAMP	ASDiv	MAWPS	TAB	MQA	MMLU STEM	SAT MATH	AVG
0	2.8	3.2	10.9	18	20.2	12.5	14.6	16.4	21.9	14.7
500	3.3	2.8	17.7	29.0	38.7	12.4	9.5	15.7	15.6	16.1
1000	4.6	4.0	18.1	31.6	41.9	15.9	11.9	18.2	25.0	19.0
1500	5.2	5.4	21.1	32.9	43.1	15.3	11.1	20.4	12.5	18.6
2000	6.8	5.8	20.2	33.5	46.6	18.2	10.7	20.3	12.5	19.4
2500	7.1	3.8	20.7	37.0	48.6	18.3	12.0	21.4	18.8	20.9
3000	7.4	4.4	22.9	37.1	50.5	18.3	12.3	21.2	25.0	22.1
3500	8.8	4.8	22.8	39.4	53.3	19.2	12.0	22.8	34.4	24.2
4000	8.6	4.6	24.0	38.7	51.4	18.8	14.8	24.4	18.8	22.7
4500	8.6	4.2	24.2	39.2	53.6	20.4	13.5	23.9	18.8	22.9
5000	8.9	5.2	24.0	40.0	52.6	20.0	13.6	23.9	18.8	23.0
5500	8.0	6.2	23.2	41.4	55.0	22.3	14.3	24.9	25.0	24.5
6000	8.3	5.2	22.2	39.8	54.0	24.3	12.6	25.1	31.2	24.7
6500	9.4	5.6	24.4	40.2	54.5	20.3	13.0	24.9	31.2	24.8
7000	9.2	5.8	25.8	40.6	55.3	22.5	12.5	24.5	21.9	24.2
7500	9.3	7.4	23.4	41.9	55.6	22.1	14.6	24.1	25.0	24.8

表 23: TinyLlama-1.1B 使用规则过滤数据的完整结果；表 24: 使用 ProX-D 数据的完整结果。

Table 25: Full evaluation results of TINYLLAMA-1.1B continual pre-training on OpenWebMath with data after ProX-D+C. Note that about 1B tokens are trained per 500 steps.

Train Steps	GSM8K	MATH	SVAMP	ASDiv	MAWPS	TAB	MQA	MMLU STEM	SAT MATH	AVG
0	2.8	3.2	10.9	18	20.2	12.5	14.6	16.4	21.9	14.7
500	4.3	5.0	16.4	28.8	36.4	15.3	11.4	18.5	15.6	16.9
1000	5.5	3.8	20.5	34.6	44.6	15.3	12.1	19.6	28.1	20.5
1500	5.2	4.4	21.4	34.5	44.7	16.1	11.2	21.4	34.4	21.5
2000	6.3	5.4	20.1	33.7	46.2	19.4	10.5	21.2	12.5	19.5
2500	7.8	5.4	22.1	37.0	49.5	17.9	13.3	22.9	21.9	22.0
3000	6.4	3.4	23.0	38.6	51.1	18.5	12.6	24.3	18.8	21.9
3500	8.5	4.6	24.1	40.2	53.8	22.1	12.5	23.1	25.0	23.8
4000	8.2	6.0	24.1	41.0	52.4	19.8	10.2	26.1	31.2	24.3
4500	8.3	5.4	24.1	41.3	54.4	20.6	15.2	24.2	28.1	24.6
5000	8.5	7.0	26.0	40.5	54.9	21.7	13.9	25.5	34.4	25.8
5500	8.7	4.0	23.2	41.1	54.8	20.5	14.4	26.5	21.9	23.9
6000	8.3	5.0	24.8	41.3	54.3	23.2	14.0	25.3	25.0	24.6
6500	8.6	6.4	24.5	41.6	55.1	22.2	14.4	26.5	25.0	24.9
7000	8.9	6.0	23.4	40.5	53.4	22.0	15.8	27.3	28.1	25.0
7500	9.0	4.4	23.8	41.9	56.4	22.2	15.6	26.8	31.2	25.7

Table 26: Full evaluation results of LLAMA-2 continual pre-training on OpenWebMath with raw data. Note that about 1B tokens are trained per 1000 steps.

Train Steps	GSM8K	MATH	SVAMP	ASDiv	MAWPS	TAB	MQA	MMLU STEM	SAT MATH	AVG
0	14.1	3.8	39.5	51.6	63.6	30.9	12.5	32.9	34.4	31.5
1k	17.2	3.6	39.1	50.4	63.0	30.2	18.9	31.8	31.2	31.7
2k	19.7	6.0	43.9	55.5	68.3	32.9	19.0	33.0	37.5	35.1
3k	19.6	8.6	42.9	56.3	68.4	32.2	17.4	34.6	40.6	35.6
4k	21.8	8.8	44.6	57.3	72.0	28.9	23.6	35.8	40.6	37.0
5k	22.6	10.4	45.9	57.0	73.5	31.5	23.9	39.0	43.8	38.6
6k	24.5	10.0	44.9	57.6	73.7	35.5	25.8	36.1	43.8	39.1
7k	23.3	10.4	46.5	59.0	75.3	32.9	27.7	39.0	50.0	40.5
8k	29.0	12.4	46.4	59.7	77.0	33.1	30.2	38.8	50.0	41.8
9k	26.1	12.8	48.8	59.9	74.3	35.0	28.3	39.2	50.0	41.6
10k	29.6	13.6	49.2	61.9	78.4	36.3	31.9	40.5	43.8	42.8

表 25: TinyLlama-1.1B 使用 ProX-D+C 数据的完整结果；表 26: Llama-2 使用原始数据的完整结果。

Table 27: Full evaluation results of LLAMA-2 continual pre-training on OpenWebMath with **ProX-D**. Note that about 1B tokens are trained per 1000 steps.

Train Steps	GSM8K	MATH	SVAMP	ASDiv	MAWPS	TAB	MQA	MMLU STEM	SAT MATH	AVG
0	14.1	3.8	39.5	51.6	63.6	30.9	12.5	32.9	34.4	31.5
1k	17.1	7.2	39.8	51.6	68.4	31.4	21.4	35.2	40.6	34.7
2k	21.9	9.2	43.2	57.0	72.8	33.1	24.0	37.6	56.2	39.4
3k	20.5	10.8	45.7	58.6	76.2	35.3	25.8	38.3	53.1	40.5
4k	27.2	11.8	45.7	58.7	76.6	35.9	29.2	41.0	31.2	39.7
5k	28.9	14.2	49.3	60.2	77.9	38.8	32.8	41.7	53.1	44.1
6k	31.9	15.0	51.5	62.0	79.0	39.2	33.3	41.4	68.8	46.9
7k	31.5	16.8	51.9	63.2	77.9	36.5	35.9	43.8	43.8	44.6
8k	30.3	13.8	51.9	63.7	80.6	38.3	36.1	41.3	59.4	46.2
9k	30.6	14.0	52.7	62.6	78.7	37.5	36.1	43.2	43.8	44.4
10k	30.3	16.0	54.2	63.8	79.5	37.3	37.2	44.2	46.9	45.5

Table 28: Full evaluation results of LLAMA-2 continual pre-training on OpenWebMath with **ProX-D+C**. Note that about 1B tokens are trained per 1000 steps.

Train Steps	GSM8K	MATH	SVAMP	ASDiv	MAWPS	TAB	MQA	MMLU STEM	SAT MATH	AVG
0	14.1	3.8	39.5	51.6	63.6	30.9	12.5	32.9	34.4	31.5
1k	18.8	6.8	40.1	54.4	66.1	29.7	22.9	35.6	53.1	36.4
2k	23.1	8.6	45.7	56.5	72.7	30.7	25.1	35.6	46.9	38.3
3k	23.4	11.8	47.9	59.1	74.6	30.4	28.2	38.3	59.4	41.5
4k	25.2	14.2	49.0	57.8	72.7	32.8	33.1	40.7	40.6	40.7
5k	24.4	13.6	48.0	58.7	72.1	28.9	33.0	40.6	50.0	41.0
6k	29.6	12.8	46.1	63.4	75.6	33.7	31.6	42.8	53.1	43.2
7k	29.9	13.6	50.5	61.5	75.2	36.4	34.5	41.7	53.1	44.0
8k	30.2	15.8	50.8	63.7	77.1	37.7	36.3	43.4	43.8	44.3
9k	34.0	15.4	52.1	62.4	79.3	35.9	40.2	44.0	56.2	46.6
10k	30.6	16.8	50.2	63.7	79.3	37.3	40.1	43.8	53.1	46.1

表 27: Llama-2 使用 ProX-D 数据的完整结果; 表 28: 使用 ProX-D+C 数据的完整结果。

Table 29: Full evaluation results of CODELLAMA-7B continual pre-training on OpenWebMath with raw data. Note that about 1B tokens are trained per 250 steps.

Train Steps	GSM8K	MATH	SVAMP	ASDiv	MAWPS	TAB	MQA	MMLU STEM	SAT MATH	AVG
0	11.8	5.0	44.2	50.7	62.6	30.6	14.3	20.4	21.9	29.1
250	16.7	8.2	45.2	52.2	65.3	33.9	16.0	28.8	43.8	34.5
500	18.3	7.8	43.1	53.9	69.0	29.3	15.3	22.5	37.5	33.0
750	20.2	8.0	45.2	54.2	71.9	29.9	17.1	31.2	37.5	35.0
1000	24.7	9.8	40.6	58.6	72.7	29.3	20.7	31.9	34.4	35.9
1250	24.3	10.4	44.0	57.5	74.8	29.2	21.4	36.1	50.0	38.6
1500	26.2	13.2	48.4	58.8	75.4	29.4	28.1	34.9	50.0	40.5
1750	25.5	11.8	49.1	58.7	76.6	32.4	26.7	37.3	43.8	40.2
2000	28.0	13.6	46.3	61.7	80.0	33.8	29.4	37.2	50.0	42.2
2250	27.7	13.6	48.9	62.2	80.3	32.5	28.9	39.1	59.4	43.6
2500	31.1	14.8	51.4	62.1	81.2	33.6	30.4	40.5	43.8	43.2

Table 30: Full evaluation results of CODELLAMA continual pre-training on OpenWebMath with **PROX-D**. Note that about 1B tokens are trained per 250 steps.

Train Steps	GSM8K	MATH	SVAMP	ASDiv	MAWPS	TAB	MQA	MMLU STEM	SAT MATH	AVG
0	11.8	5.0	44.2	50.7	62.6	30.6	14.3	20.4	21.9	29.1
250	21.1	9.2	48.7	56.1	71.3	33.4	22.2	34.1	50.0	38.5
500	23.7	11.6	49.8	57.4	74.7	32.9	28.5	35.8	59.4	41.5
750	25.1	15.4	48.1	58.9	78.8	36.8	29.4	37.6	53.1	42.6
1000	28.4	14.2	50.9	61.2	79.8	36.7	27.7	37.6	50.0	42.9
1250	33.0	15.2	49.3	62.9	81.1	33.4	32.8	41.0	46.9	44.0
1500	36.0	15.0	54.2	65.0	81.0	39.3	34.1	42.0	62.5	47.7
1750	34.7	14.6	53.1	63.6	83.3	40.6	35.9	43.4	62.5	48.0
2000	35.7	17.6	53.3	65.4	83.5	42.4	37.1	42.4	56.2	48.2
2250	37.2	18.8	54.5	65.4	83.2	41.9	41.0	44.9	71.9	51.0
2500	38.1	17.0	54.2	67.0	83.1	40.9	39.8	43.7	50.0	48.2

表 29: CodeLlama-7B 使用原始数据的完整结果; 表 30: 使用 ProX-D 数据的完整结果。

Programming Every Example: Lifting Pre-training Data Quality like Experts at Scale

Table 31: Full evaluation results of CODELLAMA continual pre-training on OpenWebMath with **PROX-D+C**. Note that about 1B tokens are trained per 250 steps.

Train Steps	GSM8K	MATH	SVAMP	ASDiv	MAWPS	TAB	MQA	MMLU STEM	SAT MATH	AVG
0	11.8	5.0	44.2	50.7	62.6	30.6	14.3	20.4	21.9	29.1
250	18.1	10.2	46.0	54.5	71.9	33.0	21.3	34.4	50.0	37.7
500	22.4	10.0	50.3	59.7	76.4	31.3	26.1	36.0	59.4	41.3
750	26.8	11.4	51.2	61.0	78.5	34.9	26.4	38.0	53.1	42.4
1000	29.0	14.4	54.1	62.8	80.1	36.9	34.2	40.4	62.5	46.0
1250	31.4	15.0	51.7	63.8	81.1	37.2	32.5	41.4	75.0	47.7
1500	31.5	17.4	53.4	64.4	80.7	39.6	35.4	41.6	71.9	48.4
1750	33.7	15.2	50.6	64.3	81.5	39.2	36.1	40.5	53.1	46.0
2000	36.2	16.0	54.7	65.1	83.1	39.9	39.1	43.4	71.9	49.9
2250	37.1	16.6	55.3	65.6	82.4	41.3	36.5	42.7	75.0	50.3
2500	35.6	17.6	55.8	67.9	82.7	41.3	38.9	42.6	62.5	49.4

Table 32: Full evaluation results of MISTRAL-7B continual pre-training on OpenWebMath with raw data. Note that about 1B tokens are trained per 1000 steps.

Train Steps	GSM8K	MATH	SVAMP	ASDiv	MAWPS	TAB	MQA	MMLU STEM	SAT MATH	AVG
0	40.6	11.4	65.4	68.5	87.0	52.9	32.3	50.0	56.2	51.6
1k	31.6	12.0	56.5	66.0	80.1	43.9	27.1	45.1	56.2	46.5
2k	32.4	10.8	54.7	63.5	82.6	40.8	31.6	45.7	59.4	46.8
3k	33.6	14.8	60.4	64.7	84.5	43.5	33.1	47.2	68.8	50.1
4k	35.1	14.8	58.7	65.2	84.4	41.2	38.5	47.3	62.5	49.7
5k	33.4	16.0	59.3	65.0	83.8	46.7	34.6	49.1	62.5	50.0
6k	38.7	16.6	61.5	68.1	86.1	47.4	35.3	48.5	37.5	48.9
7k	39.6	17.2	60.5	68.2	86.2	44.4	38.5	49.3	53.1	50.8
8k	44.0	16.4	64.5	69.8	88.7	45.5	41.3	50.6	59.4	53.4
9k	43.9	19.4	63.7	69.7	87.6	44.9	42.9	51.0	62.5	54.0
10k	44.4	19.2	65.2	69.6	88.4	46.6	43.1	50.8	65.6	54.8

表 31: CodeLlama 使用 ProX-D+C 数据的完整结果；表 32: Mistral-7B 使用原始数据的完整结果。

Table 33: Full evaluation results of MISTRAL-7B continual pre-training on OpenWebMath with **PROX-D**. Note that about 1B tokens are trained per 1000 steps.

Train Steps	GSM8K	MATH	SVAMP	ASDiv	MAWPS	TAB	MQA	MMLU STEM	SAT MATH	AVG
0	40.6	11.4	65.4	68.5	87.0	52.9	32.3	50.0	56.2	51.6
1k	36.8	14.6	57.2	66.1	83.1	45.7	32.6	47.7	59.4	49.2
2k	38.5	17.0	57.9	69.0	86.3	44.7	33.6	49.2	56.2	50.3
3k	40.0	19.0	59.3	68.7	87.0	46.8	41.0	48.0	68.8	53.2
4k	38.5	20.4	59.3	66.2	85.1	42.6	42.8	49.5	68.8	52.6
5k	42.5	20.2	63.0	70.5	86.6	47.2	43.4	49.8	62.5	54.0
6k	46.8	17.8	62.5	72.7	88.2	51.2	47.7	51.3	56.2	54.9
7k	47.5	22.4	64.1	71.8	89.1	51.4	47.9	52.4	65.6	56.9
8k	44.6	23.8	63.2	70.8	87.7	47.6	49.1	54.1	65.6	56.3
9k	46.6	24.6	61.6	72.3	86.4	46.9	49.8	53.2	65.6	56.3
10k	46.7	22.6	63.5	72.4	88.9	48.3	48.2	54.1	62.5	56.4

Table 34: Full evaluation results of Mistral-7B continual pre-training on OpenWebMath with **PROX-D+C**. Note that about 1B tokens are trained per 1000 steps.

Train Steps	GSM8K	MATH	SVAMP	ASDiv	MAWPS	TAB	MQA	MMLU STEM	SAT MATH	AVG
0	40.6	11.4	65.4	68.5	87.0	52.9	32.3	50.0	56.2	51.6
1k	30.9	16.0	60.1	64.5	85.3	40.8	33.9	48.0	59.4	48.8
2k	40.3	17.6	63.0	66.3	86.2	48.0	33.9	48.7	53.1	50.8
3k	42.4	17.8	59.6	69.1	85.7	50.1	38.5	49.9	59.4	52.5
4k	43.8	20.4	63.7	69.3	88.2	46.2	46.3	50.9	65.6	54.9
5k	42.5	18.4	59.3	69.6	87.9	44.3	46.1	51.9	65.6	54.0
6k	47.7	21.8	62.7	71.7	89.2	47.9	48.4	54.0	68.8	56.9
7k	46.8	21.6	62.9	72.1	88.4	50.1	46.4	52.5	68.8	56.6
8k	48.4	21.6	65.0	72.7	89.2	51.1	49.4	52.9	65.6	57.3
9k	48.5	24.8	64.4	72.6	88.3	50.7	48.1	53.4	62.5	57.0
10k	51.0	22.4	64.9	72.9	89.2	49.8	53.0	54.2	75.0	59.2

表 33: Mistral-7B 使用 ProX-D 数据的完整结果；表 34: 使用 ProX-D+C 数据的完整结果。

附录 F：分析

F.1 词元长度分布

方法	通用领域平均长度	数学领域平均长度
无额外精修	1217.5	1815.8
规则	1329.4	1955.6
ProX	2004.8	1734.9

如第 4 节所述，ProX 精修数据的通用领域文档长度分布发生明显变化，平均长度从 1217.5 提高到 2004.8；FineWeb 规则只提高到 1329.4。数学领域则呈相反趋势：原始平均长度 1815.8，ProX 降到 1734.9，规则提高到 1955.6。TinyLlama-1.1B 的训练性能排序是 ProX > 原始数据 > 规则，再次说明数学文档与通用网页在分布和特征上存在显著差异。

F.2 案例研究

作者通过表 36-37 定性展示 ProX 的精修效果。通用领域 RedPajama-V2 中，ProX 可以丢弃低信息文档、删除导航栏、替换 URL；数学领域中，可以删除与数学推理关系很弱的文档，并清除功能按钮等次要元素。

表 36 案例 1：整页丢弃

原文是一段娱乐性非虚构写作者 Michael Lewis 及广播节目分段介绍，虽然可读，但被文档级模型判为不符合目标质量，输出：

```
drop_doc()
```

表 36 案例 2：保留正文并清除导航/链接

网页为伯明翰大学 Dr Tim Overton 的教师介绍。ProX 保留主体，删除开头“Home > Staff > ...”等导航和联系信息，并移除 Google Scholar URL 与“Pages: 1 3 4”等模板文本：

```
keep_doc()
remove_lines(line_start=0, line_end=5)
normalize(source_str='http://scholar.google.co.uk/citations?user', target_str='')
normalize(source_str='Pages: 1 3 4', target_str='')
```

表 37 案例 1：数学论坛低质讨论

网页讨论“未杂化 π 键”，包含用户名、发帖数、加入时间，以及多位回答者明确表示“不太确定”的低置信讨论。ProX 输出 drop_doc()，删除整页。

表 37 案例 2：三角恒等式题目页面

网页正文包含“证明以下三角恒等式”及公式，但头部有 Account、Register、Share、Books Shortlist 等按钮，尾部有“必须登录查看答案”“是否有错误”等功能文本。ProX 保留文档，删除 0-7 行和 18-24 行。

```
keep_doc()
remove_lines(line_start=0, line_end=7)
```

Programming Every Example: Lifting Pre-training Data Quality like Experts at Scale

Table 36: Cases from RedPajama-V2 after applying PROX. Text in **red** indicates content to be removed or replaced. “...” denotes omitted content due to limited space.

Case 1
TagCollegeEducationJournalismWar : Michael Lewis ContributorMichael Lewis Michael Lewis is possibly the most entertaining nonfiction writer alive. If that's not true it's at least close to true. Liar's Poker, Moneyball, The Blind Side, his NYT article about Jonathan Lebed (Google it): what's not to love? 504: How I Got Into College Act Two: My Ames is True Writer Michael Lewis tells the story of a man named Emir Kamenica, whose path to college started with fleeing the war in Bosnia and becoming a refugee in the United States. Then he had a stroke of luck: a student teacher read an essay he'd plagiarized from a book he'd stolen from a library back in Bosnia, and was so impressed that she got him out of a bad high school and into a much better one. Act Three Michael Lewis' story continues, and he figures out why Emir Kamenica insists on remembering, and telling, the story of his life the way he does — even when he finds out that some of the facts may be wrong.
Output by PROX: <pre>drop_doc()</pre>
Case 2
Home ; Staff ; Staff search ; Dr Tim Overton Dr Tim Overton BSc PhD School of Chemical EngineeringSenior Lecturer Telephone (+44) (0) 121 414 5306Emailt.w.overton@bham.ac.uk AddressSchool of Chemical EngineeringUniversity of Birmingham B15 2TT Dr Tim Overton is a biochemist and molecular microbiologist who is interested in applying molecular biology and single-cell techniques to understand and develop bioprocesses. He is active in microbial flow cytometry research and collaborates widely with bioprocess engineers, molecular microbiologists, cell biologists and environmental microbiologists to develop new methods of answering fundamental questions on a single-cell level. His research also focuses on using bacteria to make useful products such as protein drugs and small molecules, and the bacterial responses to stress encountered in such processes. Current and recent research funding has come from the BBSRC, TSB and EU FP7. He is the director of the MSc in Biochemical Engineering. Pages: 1 3 4 ... Google scholar: http://scholar.google.co.uk/citations?user=tF_eBKEAAAAJ ...
Output by PROX: <pre>keep_doc() remove_lines(line_start=0, line_end=5) normalize(source_str="http://scholar.google.co.uk/citations?user", target_str="") normalize(source_str="Pages: 1 3 4", target_str="") ...</pre>

Table 37: Cases from OpenWebMath after applying PROX. Text in red indicates content to be removed or replaced. “. . .” denotes omitted content due to limited space.

Case 1
unhybridized pi bonds $sp, sp^2, sp^3, dsp^3, d^2sp^3$ Tatiana 4B Posts: 30 Joined: Fri Sep 28, 2018 12:28 am ### unhybridized pi bonds . . . ### Re: unhybridized pi bonds I am not too sure in my knowledge about this, but I think that both have hybridized orbitals. Since hybridization is defined as the phenomenon of intermixing of the orbitals such as sp, sigma and pi bonds are just different types of covalent bonds formed depending on the way the atomic orbitals hybridize with each other. Sigma bonds are a result of when the overlap of orbitals of two atoms takes place along the line joining the two orbitals, while pi bonds are when two atoms overlap due to the sideways overlap of their 'p' orbitals. Hannah Yates 1K Posts: 59 Joined: Fri Sep 28, 2018 12:27 am ### Re: unhybridized pi bonds I am also not too sure on my answer, but I am pretty sure that a sigma bond has just hybridized orbitals, but the reason a pi bond can form is because of an extra (not hybridized) p orbital. This allows for a double and triple bond to form. Output by PROX: drop_doc()
Case 2
Solution - Trigonometric Identities Account Register Share Books Shortlist ConceptTrigonometric Identities Question Prove the following trigonometric identities: (i) $\frac{\sin \theta}{1 - \cos \theta} = \operatorname{cosec} \theta + \cot \theta$ Solution You need to to view the solution Is there an error in this question or solution? Reference Material Solution for concept: Trigonometric Identities. For the course CBSE S Output by PROX: keep_doc() remove_lines(line_start=0, line_end=7) remove_lines(line_start=18, line_end=24)

原论文表 37: OpenWebMath 的 ProX 案例, 红色文字表示被删除或替换内容。

F.3 错误分析

两个精修阶段、两个领域的失败率均低于 0.5%。通用领域文档级/块级失败率分别为 0.04%/0.36%，块级程序平均 3.7 次调用；数学领域分别为 0.06%/0.11%，平均 2.7 次调用。

作者观察到两类常见失败：（1）重复输出或空输出：程序意外产生重复结果或没有有意义输出，通常与循环条件不当或处理逻辑约束不足有关；（2）删除不存在的目标：程序试图删除输入中不存在的字符串或行，在格式不规则或存在意外变化的数据中导致执行不完整或错误。

表 39 案例 1 中，模型重复生成多次 `remove_lines(start=7,end=7)`，最后一条调用还被截断；案例 2 中，模型试图删除 DOI “vol6.no2.53”，但原文实际是 “vol7.no9.159”，目标字符串不存在。

Programming Every Example: Lifting Pre-training Data Quality like Experts at Scale

Table 38: Failure ratio and average complexity (function calls) for ProX refining stages across domains.

Domain	Failure Ratio (doc-level)	Failure Ratio (chunk-level)	Complexity (AVG. function calls)
General Domain	0.04%	0.36%	3.7
Math Domain	0.06%	0.11%	2.7

Table 39: Failure cases from RedPajama-V2 during applying ProX. “...” denotes omitted content due to limited space. The notation like [004] is used to indicate the line number.

Case 1: Repeated output (or Empty output)
... [004] P: 114 1. The problem statement, all variables and given/known data Mercury is poured into a U-tube as in Figure P15.18a....Basically I don't understand why you would know to set the two volumes equal to each other? How do you know the volumes are the same? ... [007] Related Discussions Mechanical Engineering 6 Introductory Physics Homework 0 General Engineering 1 Introductory Physics Homework 2 Introductory Physics Homework 2 Output by ProX: <pre>remove_lines(start=1, end=1) remove_lines(start=6, end=6) remove_lines(start=7, end=7) remove_lines(start=7, end=7) remove_lines(start=7, end=7) remove_lines(start=7, end=7)</pre>
Case 2: Non-existent target string
... [195] 18. Sathyamoorthi, C. R., Mbekomize, C., Mapharing, M., & Selinkie, P. (2018). The Impact of Corporate Governance on Working Capital Management Efficiency: Evidence from the Listed Companies in the Consumer Services Sector in Botswana. International Journal of Economics and Finance, 10, 135. https://doi.org/10.5539/ijef.v10n12p135 [196] 19. Vu, T. M. T., Tran, C. Q., Doan, D. T., & Le, T. N. (2020). Determinants of Capital Structure: The Case in Vietnam. Journal of Asian Finance, Economics, And Business, 7(9), 159-168. https://doi.org/10.13106/jafeb.2020.vol7.no9.159 ... Output by ProX: <pre># Analysis: this 'source_str' can not be found in the original text normalize(source_str="https://doi.org/10.13106/jafeb.2020.vol6.no2.53", target_str="")</pre>

F.4 计算开销分析

根据 Kaplan 等人的近似，Transformer 语言模型的训练和推理 FLOPs 可以用非嵌入参数量 N 与词元数 D 的乘积估算：

$$C_{\text{train}} \approx 6 \cdot N \cdot D_{\text{train}} \quad (9)$$

$$C_{\text{inference}} \approx 2 \cdot N \cdot (D_{\text{prefill}} + D_{\text{decode}}) \quad (10)$$

ProX 在最终训练前进行两个数据精修阶段，因此产生额外推理 FLOPs。设每阶段精修模型参数量为 N_{refine} ，原始数据词元数为 D_{raw} 。文档级阶段：

$$\begin{aligned} C_{\text{doc}} &\approx 2 \cdot N_{\text{refine}} \cdot (D_{\text{raw}} + D_{\text{output}}) \\ &\approx 2 \cdot N_{\text{refine}} \cdot D_{\text{raw}}, \text{ 假设 } D_{\text{output}} \ll D_{\text{raw}} \end{aligned} \quad (11)$$

该阶段产生大小为 D_{doc} 的新数据池。块级阶段：

$$\begin{aligned} C_{\text{chunk}} &\approx 2 \cdot N_r \cdot (D_{\text{doc}} + D_{\text{output}}) \\ &\approx 2 \cdot N_r \cdot D_{\text{doc}}, \text{ 假设 } D_{\text{output}} \ll D_{\text{doc}} \end{aligned} \quad (12)$$

最终得到 D_{ProX} 。总开销为：

$$\begin{aligned} C_{\text{ProX}} &= C_{\text{doc}} + C_{\text{chunk}} \\ &\approx 2 \cdot N_{\text{doc_refine}} \cdot D_{\text{raw}} + 2 \cdot N_{\text{chunk_refine}} \cdot D_{\text{doc}} \end{aligned} \quad (13)$$

通常两个阶段使用相同尺寸的精修模型，因此可简化为：

$$C_{\text{ProX}} \approx 2 \cdot N_{\text{refine}} \cdot (D_{\text{raw}} + D_{\text{doc}}) \quad (14)$$

作者省略了微调 FLOPs，因为相较于大规模预训练与推理，该部分可以忽略。

译文结束

本译本完整覆盖正式论文的正文、附录叙述、提示词、公式、主要表格和全部附录数值表页面。参考文献和大规模数值表中的专有名词、缩写和数值按原文保留。